

HASTANE YÖNETİM SİSTEMİ

Programlama II – Dönem Projesi Raporu

Teknolojiler: .NET MAUI · ASP.NET Core Minimal API · Entity Framework Core · SQLite / SQL Server · JWT

Mayıs 2026

Hazırlayanlar

Eftalya Beril ŞAHİN-2212721037

Azra KAYA- 2112721005

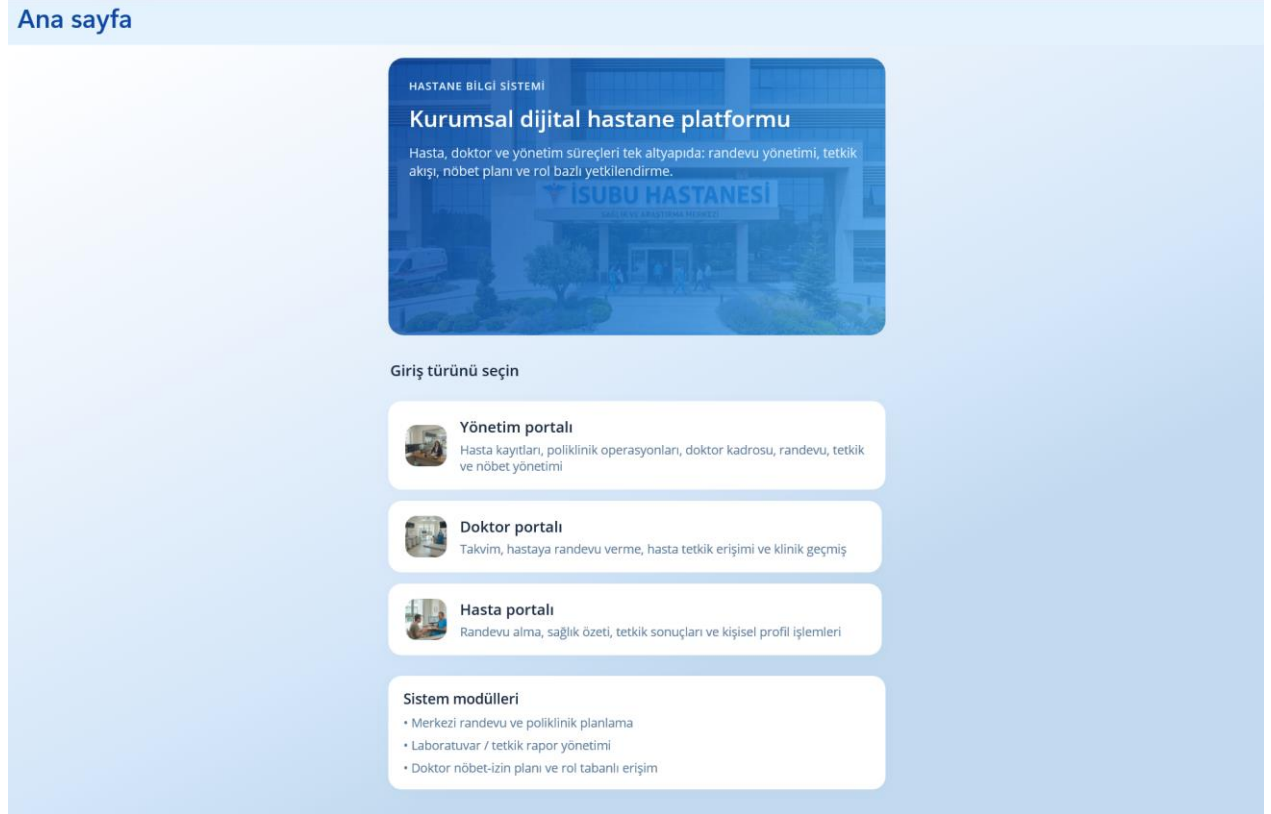
1. Proje Genel Tanımı

Bu proje, gerçek bir hastane ortamını dijital olarak yönetmek amacıyla geliştirilmiş kapsamlı bir Hastane Yönetim Sistemi'dir. Sistemin temel amacı; hasta kayıtlarını tutmak, poliklinik ve doktor bilgilerini yönetmek, randevu planlaması yapmak, tetkik raporlarını saklamak ve doktor nöbet/izin takibini gerçekleştirmektir.

Sistem üç ana yazılım bileşeninden oluşmaktadır:

- **Hospital.Client:** .NET MAUI ile geliştirilmiş masaüstü/mobil istemci uygulaması
- **Hospital.Api:** ASP.NET Core Minimal API ile geliştirilmiş RESTful backend sunucusu
- **Hospital.Shared:** İstemci ve API arasında paylaşılan DTO sınıfları ve enum tanımları

Kullanıcılar üç farklı role sahiptir: Yönetici (Admin), Doktor ve Hasta. Her rol, sisteme kendi portalı üzerinden giriş yapar ve yalnızca kendi yetkisi dahilindeki işlemleri gerçekleştirebilir. Kimlik doğrulama mekanizması JWT (JSON Web Token) tabanlıdır; token 4 saat geçerlidir. Veritabanı olarak geliştirme ortamında SQLite, üretim ortamında SQL Server kullanılabilir — bu seçim tek bir yapılandırma bayrağı ile kontrol edilir.



Şekil1: Hastane Yönetim Portalı'na giriş

Yönetim girişi



HASTANE BİLGİ SİSTEMİ

Kurumsal giriş

Hoş geldiniz

Hasta kayıtları, poliklinikler, doktorlar ve randevu yönetimi için güvenli oturum açın.

Kullanıcı adı

admin

Şifre

••••••••

Göster

☒ Kullanım koşullarını ve kişisel verilerin korunmasına ilişkin aydınlatmayı okudum, kabul ediyorum.
[Kullanıcı sözleşmesini oku](#)

Güvenli giriş

Şekil2: Admin portalına giriş aşaması

Doktor girişi



HASTANE BİLGİ SİSTEMİ

Doktor portalı

Hoş geldiniz

Yalnızca kendi randevu listenize erişirsiniz; yönetim işlemleri kurumsal hesaptadır.

Portal kullanıcı adı

dr.ayilmaz

Şifre

••••••••

Göster

☒ Kullanım koşullarını ve kişisel verilerin korunmasına ilişkin aydınlatmayı okudum, kabul ediyorum.
[Kullanıcı sözleşmesini oku](#)

Güvenli giriş

Şekil3: Doktor portalına giriş aşaması

İSTER 1 – Proje .NET MAUI ile Yapılmış mı?

Proje tamamen .NET MAUI (Multi-platform App UI) teknolojisi kullanılarak geliştirilmiştir. İstemci uygulaması olan Hospital.Client, MAUI mimarisinin tüm gereksinimlerini karşılar: XAML tabanlı kullanıcı arayüzü, platform-bağımsız proje yapısı ve MauiApp başlangıç modeli.

Proje dosyasında `<UseMaui>true</UseMaui>` tanımı bulunmaktadır; bu ifade, projenin bir .NET MAUI uygulaması olduğunu derleyiciye ve dağıtım araçlarına bildiren temel direktiftir. Uygulama `MauiProgram.cs` dosyası üzerinden başlatılmakta; burada dependency injection konteyneri, font kaynakları ve servis kayıtları yapılandırılmaktadır. Hedef framework olarak `net10.0-windows10.0.19041.0` tanımlıdır; iOS ve macOS Catalyst hedefleri de proje dosyasında listelenmiştir. Sayfalar (Views), ViewModels ve Shell navigasyon yapısı MAUI mimarisine uygun biçimde düzenlenmiştir.

Kod: `src/Hospital.Client/Hospital.Client.csproj`

```
Hospital.Client HospitalEndpoints.cs
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <!-- Android: makinede Android SDK varsa aşağıdaki net10.0-android hedefini TargetFrameworks'e ekleyin. -->
    <TargetFrameworks>net10.0-ios;net10.0-maccatalyst</TargetFrameworks>
    <TargetFrameworks Condition="$([MSBuild]::IsOSPlatform('windows'))">$(TargetFrameworks);net10.0-windows10.0.19041.0</TargetFramework
```

Kod: `src/Hospital.Client/MauiProgram.cs`

```
MauiProgram.cs Hospital.Client HospitalEndpoints.cs
Hospital.Client (net10.0-ios) Hospital.Client.MauiProgram CreateMauiApp()

4
5 namespace Hospital.Client;
6
7 3 başvuru
8 public static class MauiProgram
9 {
10     3 başvuru
11     public static MauiApp CreateMauiApp()
12     {
13         var builder = MauiApp.CreateBuilder();
14         builder
15             .UseMauiApp<App>()
16             .ConfigureFonts(fonts =>
17             {
18                 fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
19                 fonts.AddFont("OpenSans-Semibold.ttf", "OpenSansSemibold");
20             });
21     }
22 }
```

İSTER 2 – Minimal API Projesi Oluşturulmuş mu?

Backend tarafında HospitalApi adında bir ASP.NET Core Web projesi oluşturulmuştur. Bu proje, geleneksel Controller/Action yapısını kullanmak yerine ASP.NET Core 7+'da sunulan Minimal API yaklaşımını benimsemektedir. Minimal API'de her endpoint; MapGet, MapPost, MapPut veya MapDelete metodu ile doğrudan bir lambda fonksiyonuna ya da metoda bağlanır. Bu sayede daha az şablon kodu yazılır ve her uç noktanın sorumluluğu tek bir yerde toplanır.

Tüm yönetici uç noktaları HospitalEndpoints.cs dosyasında, hasta portal uç noktaları ise PortalEndpoints.cs dosyasında tanımlanmıştır. Her iki dosya da MapHospitalEndpoints() ve MapPortalEndpoints() extension metotları ile Program.cs'e bağlanmaktadır. Bu yaklaşım, uç noktaları feature bazlı gruplara ayırarak okunabilirliği artırır. Toplam endpoint sayısı 40'ı aşmaktadır: hasta, klinik, doktor, randevu, lab raporu, doktor görevi ve kimlik doğrulama işlemleri için ayrı ayrı MapGet/MapPost/MapPut/MapDelete çağrıları mevcuttur.

Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs (seçme örnekler)

```
6 namespace HospitalApi.Endpoints;
7
8 public static class HospitalEndpoints
9 {
10     public static void MapHospitalEndpoints(this WebApplication app)
11     {
12         app.MapPost("/api/auth/login", async (LoginRequest request, IAuthService authService)
13         {
14             var validation = RequestValidator.Validate(request);
15             if (validation is not null)
16             {
17                 return validation;
18             }
19
20             var result = await authService.LoginAsync(request);
21             return result is null ? Results.Unauthorized() : Results.Ok(result);
22         }).AllowAnonymous();
23
24         app.MapPost("/api/auth/login/doctor", async (DoctorPortalLoginRequest request, IAuthService authService)
25         {
26             var validation = RequestValidator.Validate(request);
27             if (validation is not null)
28             {
29                 return validation;
30             }
31
32             var result = await authService.LoginDoctorPortalAsync(request);
33             return result is null ? Results.Unauthorized() : Results.Ok(result);
34         }).AllowAnonymous();
35
36         app.MapPost("/api/auth/login/patient", async (PatientPortalLoginRequest request, IAuthService authService)
37         {
38             var validation = RequestValidator.Validate(request);
39             if (validation is not null)
40             {
41                 return validation;
42             }
43
44             var result = await authService.LoginPatientPortalAsync(request);
45             return result is null ? Results.Unauthorized() : Results.Ok(result);
46         }).AllowAnonymous();
47
48         app.MapPut("/api/auth/change-password", async (ChangePasswordRequest request, IAuthService authService)
49         {
50             var validation = RequestValidator.Validate(request);
51             if (validation is not null)
52             {
53                 return validation;
54             }
55
56             var id = GetUserId(user);
57             if (id is null)
58             {
59                 return Results.Unauthorized();
60             }
61
62             var (success, error) = await authService.ChangePasswordAsync(id.Value, request);
```

```
67
68 api.MapGet("/patients", async (string? search, string? nationalId, IPatientService
69 {
70     var list = await patients.GetAsync(new PatientListQuery { Search = search, Nati
71     return Results.Ok(list);
72 });
73
74 api.MapGet("/patients/{id:int}", async (int id, IPatientService patients) =>
75 {
76     var item = await patients.GetByIdAsync(id);
77     return item is null ? Results.NotFound() : Results.Ok(item);
78 });
79
80 api.MapPost("/patients", async (CreatePatientRequest request, IPatientService patie
81 {
82     var validation = RequestValidator.Validate(request);
83     if (validation is not null)
84     {
85         return validation;
86     }
87
88     var (success, error, patient) = await patients.CreateAsync(request);
89     return success ? Results.Created($"/api/patients/{patient!.Id}", patient) : Res
90 });
91
92 api.MapPut("/patients/{id:int}", async (int id, UpdatePatientRequest request, IPati
93 {
94     var validation = RequestValidator.Validate(request);
95     if (validation is not null)
96     {
97         return validation;
98     }
99
100     var (success, error) = await patients.UpdateAsync(id, request);
101     return success ? Results.NoContent() : Results.BadRequest(new { error });
102 });
103
104 api.MapDelete("/patients/{id:int}", async (int id, IPatientService patients) =>
```

İSTER 3 – Entity Framework Core Code First Yaklaşımı

Veritabanı erişimi Entity Framework Core (EF Core) kütüphanesi ile sağlanmaktadır ve Code First yaklaşımı benimsenmiştir. Code First yaklaşımında veritabanı şeması; SQL sorguları yazılarak değil, C# sınıfları (entity'ler) tanımlanarak oluşturulur. EF Core bu sınıfları inceleyerek gerekli tablo ve sütunları otomatik üretir.

HospitalDbContext sınıfı, DbContext'ten türer ve her tablo için bir DbSet<T> özelliği içerir. OnModelCreating metodu içinde Fluent API kullanılarak tablo adları, birincil anahtar tanımları, zorunlu alan kısıtlamaları (IsRequired), maksimum uzunluklar (HasMaxLength) ve yabancı anahtar ilişkileri (HasOne/WithMany) tanımlanmaktadır. Bu yapı sayesinde veritabanı şeması tamamen C# kodu ile kontrol altında tutulur.

Kod: src/HospitalApi/Data/HospitalDbContext.cs

```
HospitalApi
HospitalApi.Data.HospitalDbContext
HospitalDbContext(DbContextOptions<HospitalDbContext> options)

1 using HospitalApi.Entities;
2 using Microsoft.EntityFrameworkCore;
3
4 namespace HospitalApi.Data;
5
6 public sealed class HospitalDbContext : DbContext
7 {
8     public HospitalDbContext(DbContextOptions<HospitalDbContext> options)
9         : base(options)
10     {
11     }
12
13     public DbSet<AppUser> Users => Set<AppUser>();
14     public DbSet<Patient> Patients => Set<Patient>();
15     public DbSet<Clinic> Clinics => Set<Clinic>();
16     public DbSet<Doctor> Doctors => Set<Doctor>();
17     public DbSet<Appointment> Appointments => Set<Appointment>();
18     public DbSet<LabReport> LabReports => Set<LabReport>();
19     public DbSet<DoctorDuty> DoctorDuties => Set<DoctorDuty>();
20
21     protected override void OnModelCreating(ModelBuilder modelBuilder)
22     {
23         modelBuilder.Entity<AppUser>(e =>
24         {
25             e.HasKey(x => x.Id);
26             e.HasIndex(x => x.UserName).IsUnique();
27             e.Property(x => x.UserName).HasMaxLength(64).IsRequired();
28             e.Property(x => x.PasswordHash).HasMaxLength(500).IsRequired();
29             e.Property(x => x.FullName).HasMaxLength(200).IsRequired();
30             e.Property(x => x.Role).HasMaxLength(50).IsRequired();
31         });
32     }
33 }
```

Kod: src/HospitalApi/Entities/Patient.cs

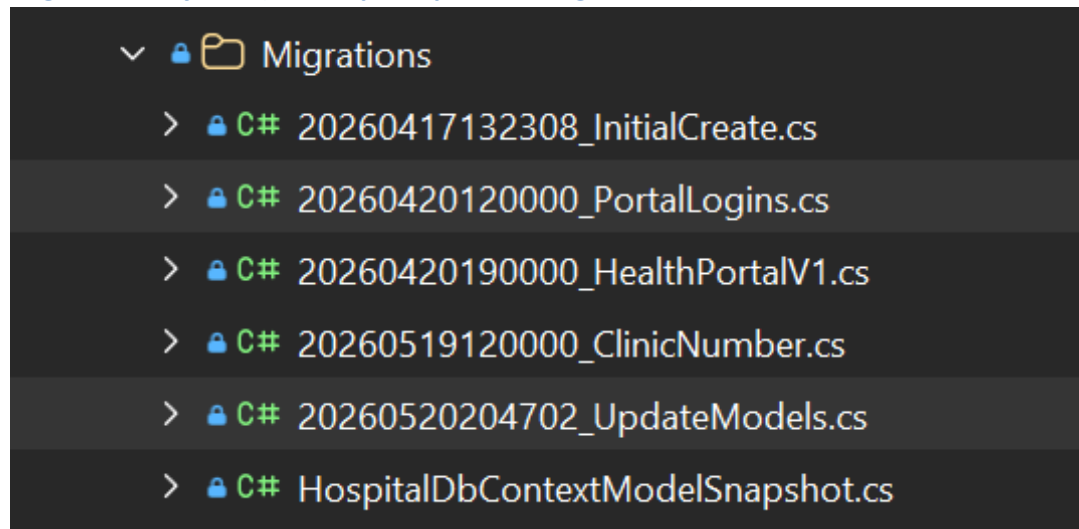
```
HospitalApi HospitalApi.Entities.Patient FirstName
1 namespace HospitalApi.Entities;
2
3 public sealed class Patient : AuditableEntity
4 {
5     public string FirstName { get; set; } = string.Empty;
6     public string LastName { get; set; } = string.Empty;
7     public string NationalId { get; set; } = string.Empty;
8     public string Phone { get; set; } = string.Empty;
9     public string? Email { get; set; }
10    public DateTime BirthDate { get; set; }
11
12    public string? EmergencyContactName { get; set; }
13    public string? EmergencyContactPhone { get; set; }
14    public string? EmergencyContactPhone { get; set; }
15
16    /// <summary>Hasta portal şifresi; boşsa hasta girişi kapalıdır.</summary>
17    public string? PortalPasswordHash { get; set; }
18
19    public ICollection<Appointment> Appointments { get; set; } = new List<Appointment>();
20
21    public ICollection<LabReport> LabReports { get; set; } = new List<LabReport>();
22 }
```

İSTER 4 – Migration'lar Eklenmiş mi?

EF Core migration sistemi, veritabanı şemasını zaman içinde evrimleştirmek için kullanılır. Her migration; bir önceki durumdan yeni duruma geçişi tanımlayan Up() ve geri almayı tanımlayan Down() metodlarını içerir. Böylece takım üyeleri veritabanını tutan SQL betiklerini elle yazmak zorunda kalmaz; migration komutları yeterlidir.

Projede beş adet migration bulunmaktadır. İlk migration (InitialCreate) temel tabloları oluşturur. Sonraki migration'lar portal giriş alanları, sağlık portalı genişlemesi, klinik numarası ve model güncellemelerini kapsamaktadır. Uygulama başlangıcında DbInitializer.SeedAsync() çağrısı, migration'ları uygulayarak demo verileri de seed eder.

Migration Dosyaları (src/HospitalApi/Data/Migrations/)



Kod: 20260417132308_InitialCreate.cs

```
20260417132...ialCreate.cs X 20260519120...nicNumber.cs
HospitalApi HospitalApi.Data.Migrations.InitialCre: Up(MigrationBuilder migrationBuilder)
1 using System;
2 using Microsoft.EntityFrameworkCore.Migrations;
3
4 #nullable disable
5
6 namespace HospitalApi.Data.Migrations
7 {
8     /// <inheritdoc />
9     1 başvuru
10     public partial class InitialCreate : Migration
11     {
12         /// <inheritdoc />
13         0 başvuru
14         protected override void Up(MigrationBuilder migrationBuilder)
15         {
16             namespace HospitalApi.Data.Migrations
17             public partial class InitialCreate
18             protected override void Up(MigrationBuilder migrationBuilder)
19             migrationBuilder.CreateTable(
20
21                 Id = table.Column<int>(type: "int", nullable: false)
22                 .Annotation("SqlServer:Identity", "1, 1"),
23                 Name = table.Column<string>(type: "nvarchar(200)", maxLength: 200, null
24                 Description = table.Column<string>(type: "nvarchar(500)", maxLength: 500, null
25                 CreatedAt = table.Column<DateTime>(type: "datetime2", nullable: false),
26                 CreatedByUserId = table.Column<int>(type: "int", nullable: true),
27                 UpdatedAt = table.Column<DateTime>(type: "datetime2", nullable: true),
28                 UpdatedByUserId = table.Column<int>(type: "int", nullable: true)
29             },
30             constraints: table =>
31             {
32                 table.PrimaryKey("PK_Clinics", x => x.Id);
33             });
34
35             migrationBuilder.CreateTable(
36                 name: "Patients",
37                 columns: table => new
38                 {
39                     Id = table.Column<int>(type: "int", nullable: false)
40                     .Annotation("SqlServer:Identity", "1, 1"),
41                     FirstName = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
```

İSTER 5 – Veritabanı Bağlantısı Yapılmış mı?

API projesi hem SQL Server hem de SQLite veritabanı motorlarını desteklemektedir. Hangi motorun kullanılacağı, appsettings.json dosyasındaki 'UseSqlite' boolean bayrağı ile belirlenir. Bu yaklaşım; geliştiricilerin Docker veya SQL Server kurulumuna ihtiyaç duymadan SQLite ile kolayca çalışmasını, üretim ortamında ise SQL Server'a geçişi yapılandırma değişikliğiyle sağlamasını mümkün kılar.

SQL Server bağlantısı için connection string, appsettings.Development.json dosyasından okunur. SQLite modunda ise veritabanı dosyası (hospital.db), uygulamanın çalışma dizininde (AppContext.BaseDirectory) otomatik olarak oluşturulur. Her iki seçenek de EF Core'un ilgili DbContextOptions builder metotları ile yapılandırılır.

Kod: src/HospitalApi/Program.cs

```
14 var builder = WebApplication.CreateBuilder(args);
15
16 var useSqlite = builder.Configuration.GetValue<bool>("UseSqlite");
17
18 if (useSqlite)
19 {
20     var sqlitePath = Path.Combine(AppContext.BaseDirectory, "hospital.db");
21     builder.Services.AddDbContext<HospitalDbContext>(options =>
22         options.UseSqlite($"Data Source={sqlitePath}"));
23 }
24 else
25 {
26     var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
27         ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found");
28     builder.Services.AddDbContext<HospitalDbContext>(options =>
29         options.UseSqlServer(connectionString));
30 }
```

Kod: src/HospitalApi/appsettings.json

```
1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=HospitalDb;Trusted_Connection=true;Password=;User=sa"
4   },
5   "Jwt": {
6     "Issuer": "HospitalApi",
7     "Audience": "HospitalClient",
8     "SigningKey": "ChangeThisDevelopmentKey_32chars_minimum!!",
9     "ExpireMinutes": 240
10  },
11  "Logging": {
12    "LogLevel": {
13      "Default": "Information",
14      "Microsoft.AspNetCore": "Warning"
15    }
16  },
17  "AllowedHosts": "*"
18 }
19 }
```

Kod: src/HospitalApi/Properties/launchSettings.json – SQLite profili

```
19  },
20  },
21  "http-sqlite": {
22    "commandName": "Project",
23    "dotnetRunMessages": true,
24    "launchBrowser": true,
25    "applicationUrl": "http://localhost:5059",
26    "environmentVariables": {
27      "ASPNETCORE_ENVIRONMENT": "Development",
28      "UseSqlite": "true"
29    }
30  },
31  "http-localdb": {
32    "commandName": "Project",
33    "dotnetRunMessages": true,
34    "launchBrowser": true,
35    "applicationUrl": "http://localhost:5059",
36    "environmentVariables": {
37      "ASPNETCORE_ENVIRONMENT": "Development",
38      "ConnectionStrings__DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=Hosp
39    }
40  },
41  "https": {
42    "commandName": "Project",
43    "dotnetRunMessages": true,
44    "launchBrowser": true,
45    "applicationUrl": "https://localhost:7197;http://localhost:5059",
46    "environmentVariables": {
47      "ASPNETCORE_ENVIRONMENT": "Development"
48    }
49  },
50  "IIS Express": {
51    "commandName": "IISExpress",
52    "launchBrowser": true,
53    "environmentVariables": {
54      "ASPNETCORE_ENVIRONMENT": "Development"
55    }
56  }
```

İSTER 6 – Veritabanından Listeleme ve Sorgulama

Sistemdeki tüm ana varlıklar (hasta, doktor, klinik, randevu, lab raporu, doktor görevi) veritabanından listelenerek MAUI ekranlarına yansıtılmaktadır. Her listeleme işlemi filtreleme parametreleri içerir; bu sayede kullanıcılar geniş bir liste içinden aradıklarını kolayca bulabilir.

Hasta listesinde ad, soyad, telefon, e-posta veya T.C. kimlik numarasına göre arama yapılabilir. Randevu listesinde klinik, doktor, tarih aralığı (tümü/bugün/bu hafta) ve randevu durumuna (Planlandı/Tamamlandı/İptal) göre filtreleme mevcuttur. Doktor listesi kliniğe göre filtrelenebilir. Tüm bu sorgular sunucu tarafında EF Core LINQ sorguları ile yapılmakta, gereksiz kayıtlar veritabanı düzeyinde elenerek ağ trafiği minimize edilmektedir.

Hasta listesi

Ad, telefon veya T.C. ile arayın; satıra dokunarak kaydı açın. Listeyi yenilemek için aşağı çekin.

+ Yeni hasta kaydı oluşturun

Arama

Ad, soyad, telefon veya e-posta

T.C. kimlik (tam eşleşme)

Ara

Eftalya Beril Şahin

11173221086

05551112233

15.06.2000

Ali Veli

12345678901

05550001122

15.01.1990

Şekil4 : Veritabanındaki kayıtlı hastalar Ad,Soyad, telefon veya e-posta ile aratılabilmektedir.

Randevu planı

Hasta, birim ve hekim seçin; tarih-saat ile durumu güncelleyin

Hasta

☐

Yeni hasta (kayıt yok)

Hasta seçin

Eftalya Beril Şahin

Doktor

Doktor seçin

Ayşe Yılmaz — Dahiliye

Zamanlama

Tarih

1.06.2026

Saat

Saat seçin

101 — Dahiliye

102 — Kardiyoloji

103 — Ortopedi

104 — Göz

105 — KBB

106 — Nöroloji

107 — Pediatri

108 — Üroloji

109 — Dermatoloji

110 — Genel Cerrahi

Hafta içi 08:30–16:30, öğle 12:00–13:00 hariç, 15 dk aralıklar. Gri slot'lar dolu randevu gösterir.

Durum

Durum seçin

Planlandı

Notlar

Şekil5: Veritabanında yer alan poliklinik kayıtlarına göre hastaya randevu verilebilmektedir.

Kod: src/HospitalApi/Services/PatientService.cs – LINQ filtreleme zinciri

```
10 public sealed class PatientService(HospitalDbContext db, ICurrentUserContext currentUser, I
11 {
12     3 başvuru
13     public async Task<IReadOnlyList<PatientDto>> GetAsync(PatientListQuery query, Cancellat
14     {
15         var linq = db.Patients.AsNoTracking().AsQueryable();
16
17         if (!string.IsNullOrEmpty(query.Search))
18         {
19             var term = query.Search.Trim();
20             linq = linq.Where(p =>
21                 p.FirstName.Contains(term) ||
22                 p.LastName.Contains(term) ||
23                 (p.Email != null && p.Email.Contains(term)) ||
24                 p.Phone.Contains(term));
25
26         if (!string.IsNullOrEmpty(query.NationalId))
27         {
28             var nid = query.NationalId.Trim();
29             linq = linq.Where(p => p.NationalId == nid);
30         }
31
32         return await linq
33             .OrderByDescending(p => p.CreatedAt)
34             .Select(p => new PatientDto
35             {
36                 Id = p.Id,
37                 FirstName = p.FirstName,
38                 LastName = p.LastName,
39                 NationalId = p.NationalId,
40                 Phone = p.Phone,
41                 Email = p.Email,
42                 BirthDate = p.BirthDate,
43                 EmergencyContactName = p.EmergencyContactName,
44                 EmergencyContactPhone = p.EmergencyContactPhone,
45                 EmergencyContactRelation = p.EmergencyContactRelation,
46                 CreatedAt = p.CreatedAt,
47                 CreatedByUserId = p.CreatedByUserId,
```

İSTER 7 – Veritabanına Veri Ekleme

Sisteme yeni kayıt eklemek için her varlık türüne özel bir POST endpoint'i ve servis metodu mevcuttur. MAUI istemcisinde her liste sayfasında bir 'Yeni Ekle' butonu bulunur; bu butona basıldığında ilgili düzenleme sayfası boş açılır. Kullanıcı formu doldurup 'Kaydet'e bastığında istemci, API'ye POST isteği gönderir.

Ekleme işleminde önce istek nesnesi (DTO) doğrulanır; zorunlu alanlar boş ise hata döndürülür. Doğrulama geçilirse entity nesnesi oluşturulur, CreatedAt ve CreatedByUserId audit alanları doldurulur, veritabanına eklenir ve değişiklikler kaydedilir. Başarılı ekleme sonucunda HTTP 201 Created yanıtı döner; hata durumunda HTTP 400 Bad Request ve hata mesajı döner.

Hasta kaydı

Kimlik ve iletişim bilgilerini güncel tutun; T.C. kimlik numarası resmi kayıt için kullanılır.

Temel bilgiler

Ad

Soyad

T.C. kimlik numarası

Telefon

E-posta

Doğum tarihi

Kaydet

Şekil6: Hasta kaydı ekranı

Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs – POST /patients

```
79
80 api.MapPost("/patients", async (CreatePatientRequest request, IPatientService patient
81 {
82     var validation = RequestValidator.Validate(request);
83     if (validation is not null)
84     {
85         return validation;
86     }
87
88     var (success, error, patient) = await patients.CreateAsync(request);
89     return success ? Results.Created($"api/patients/{patient!.Id}", patient) : Resul
90 });
91
92 api.MapPut("/patients/{id:int}", async (int id, UpdatePatientRequest request, IPatien
93 {
94     var validation = RequestValidator.Validate(request);
95     if (validation is not null)
96     {
97         return validation;
98     }
99
100     var (success, error) = await patients.UpdateAsync(id, request);
101     return success ? Results.NoContent() : Results.BadRequest(new { error });
102 });
103
104 api.MapDelete("/patients/{id:int}", async (int id, IPatientService patients) =>
```

Kod: src/HospitalApi/Services/PatientService.cs – CreateAsync

```
2 başvuru
1107 public async Task<(bool Success, string? Error, PatientDto? Patient)> CreateAsync(CreatePatientRequest request, CancellationTok
1108 {
1109     var exists = await db.Patients.AnyAsync(p => p.NationalId == request.NationalId, cancellationToken);
1110     if (exists)
1111     {
1112         return (false, "Bu T.C. kimlik numarası ile kayıtlı hasta zaten var.", null);
1113     }
1114
1115     var userId = currentUser.UserId;
1116     var entity = new Patient
1117     {
1118         FirstName = request.FirstName.Trim(),
1119         LastName = request.LastName.Trim(),
1120         NationalId = request.NationalId.Trim(),
1121         Phone = request.Phone.Trim(),
1122         Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim(),
1123         BirthDate = request.BirthDate.Date,
1124         CreatedAt = DateTime.UtcNow,
1125         CreatedByUserId = userId
1126     };
1127     entity.PortalPasswordHash = patientPasswordHasher.HashPassword(entity, "Patient123!");
1128
1129     db.Patients.Add(entity);
1130     await db.SaveChangesAsync(cancellationToken);
1131
1132     var dto = await GetByIdAsync(entity.Id, cancellationToken);
1133     return (true, null, dto);
1134 }
1135
1136 2 başvuru
1107 public async Task<(bool Success, string? Error)> UpdateAsync(int id, UpdatePatientRequest request, CancellationTok
1108 {
1109     var entity = await db.Patients.FirstOrDefaultAsync(p => p.Id == id, cancellationToken);
1110     if (entity is null)
1111     {
1112         return (false, "Hasta bulunamadı.", null);
1113     }
1114     entity.FirstName = request.FirstName.Trim();
1115     entity.LastName = request.LastName.Trim();
1116     entity.NationalId = request.NationalId.Trim();
1117     entity.Phone = request.Phone.Trim();
1118     entity.Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim();
1119     entity.BirthDate = request.BirthDate.Date;
1120     entity.CreatedAt = DateTime.UtcNow;
1121     entity.CreatedByUserId = currentUser.UserId;
1122     entity.PortalPasswordHash = patientPasswordHasher.HashPassword(entity, "Patient123!");
1123
1124     db.Patients.Update(entity);
1125     await db.SaveChangesAsync(cancellationToken);
1126
1127     var dto = await GetByIdAsync(entity.Id, cancellationToken);
1128     return (true, null, dto);
1129 }
```

İSTER 8 – Veritabanındaki Veri Üzerinde Güncelleme

Mevcut kayıtlar güncellenebilir. MAUI istemcisinde listeden bir kayıt seçildiğinde düzenleme sayfası mevcut verilerle dolu açılır. Kullanıcı istediği alanı değiştirip 'Kaydet'e bastığında istemci, API'ye PUT isteği gönderir.

Güncelleme işleminde sunucu tarafında güncellenen kayıt önce veritabanından bulunur; bulunamazsa 404 Not Found döner. Kayıt bulunursa gelen istek verileri entity üzerine uygulanır, UpdatedAt ve UpdatedByUserId audit alanları güncellenir ve değişiklikler kaydedilir. Başarılı güncelleme HTTP 204 No Content ile bildirilir. Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs – PUT /patients/{id}

```
api.MapPut("/patients/{id:int}", async (int id, UpdatePatientRequest request, IPatientService patients) :
{
    var validation = RequestValidator.Validate(request);
    if (validation is not null)
    {
        return validation;
    }

    var (success, error) = await patients.UpdateAsync(id, request);
    return success ? Results.NoContent() : Results.BadRequest(new { error });
});

api.MapDelete("/patients/{id:int}", async (int id, IPatientService patients) =>
{
    var (success, error) = await patients.DeleteAsync(id);
    return success ? Results.NoContent() : Results.BadRequest(new { error });
});
```

Kod: src/HospitalApi/Services/PatientService.cs – UpdateAsync

```
21 aşvuru
public async Task<(bool Success, string? Error)> UpdateAsync(int id, UpdatePatientRequest request, CancellationT
{
    var entity = await db.Patients.FirstOrDefaultAsync(p => p.Id == id, cancellationToken);
    if (entity is null)
    {
        return (false, "Hasta bulunamadı.");
    }

    var duplicate = await db.Patients.AnyAsync(
        p => p.NationalId == request.NationalId && p.Id != id,
        cancellationToken);
    if (duplicate)
    {
        return (false, "Bu T.C. kimlik numarası başka bir hastaya ait.");
    }

    entity.FirstName = request.FirstName.Trim();
    entity.LastName = request.LastName.Trim();
    entity.NationalId = request.NationalId.Trim();
    entity.Phone = request.Phone.Trim();
    entity.Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim();
    entity.BirthDate = request.BirthDate.Date;
    entity.UpdatedAt = DateTime.UtcNow;
    entity.UpdatedByUserId = currentUser.UserId;

    await db.SaveChangesAsync(cancellationToken);
    return (true, null);
}
```

Hasta kaydı

Kimlik ve iletişim bilgilerini güncel tutun; T.C. kimlik numarası resmi kayıt için kullanılır.

Ad	Soyad
Ali	Veli
T.C. kimlik numarası	
12345678901	
Telefon	E-posta
05550001122	İsteğe bağlı
Doğum tarihi	
15.01.1990 	
Kaydet	
Kaydı sil	

Şekil7: Hasta güncelleme ekranı. İstenilirse hasta kaydı bu ekrandan silinebilir.

İSTER 9 – Veritabanındaki Veri Silme

Kayıtlar silinebilir; ancak silme işlemi iş kuralları kontrolleriyle korunmaktadır. MAUI istemcisinde düzenleme sayfasında veya liste sayfasında 'Sil' butonu bulunur. Butona basıldığında 'Emin misiniz?' onay diyalogu gösterilir. Onay verildiğinde istemci API'ye DELETE isteği gönderir.

Sunucu tarafında referans bütünlüğü kontrol edilir. Örneğin randevusu bulunan bir hasta silinemez; bu durumda açıklayıcı bir hata mesajı döner. Benzer şekilde doktoru olan bir klinik veya aktif randevusu olan bir doktor silme işlemine izin vermez. Silme başarılıysa HTTP 204 No Content döner. MAUI tarafında hata mesajı DisplayAlert ile kullanıcıya gösterilir.

Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs – DELETE /patients/{id}

```
api.MapDelete("/patients/{id:int}", async (int id, IPatientService patients) =>
{
    var (success, error) = await patients.DeleteAsync(id);
    return success ? Results.NoContent() : Results.BadRequest(new { error });
});
```

Kod: src/HospitalApi/Services/PatientService.cs – DeleteAsync

```
2 başvuru
public async Task<(bool Success, string? Error)> DeleteAsync(int id, CancellationToken cancellationToken = d
{
    var entity = await db.Patients.Include(p => p.Appointments).FirstOrDefaultAsync(p => p.Id == id, cancell
    if (entity is null)
    {
        return (false, "Hasta bulunamadı.");
    }

    if (entity.Appointments.Any())
    {
        return (false, "Randevusu olan hasta silinemez.");
    }

    db.Patients.Remove(entity);
    await db.SaveChangesAsync(cancellationToken);
    return (true, null);
}
```

Kod: src/Hospital.Client/PatientsEditPage.xaml.cs – MAUI silme onay diyalogu

```
0 başvuru
private async void OnDeleteClicked(object? sender, EventArgs e)
{
    if (_patientId is not > 0)
    {
        return;
    }

    if (!await DisplayAlert("Sil", "Bu hastayı silmek istediğinize emin misiniz?", "Evet", "Hayır"))
    {
        return;
    }

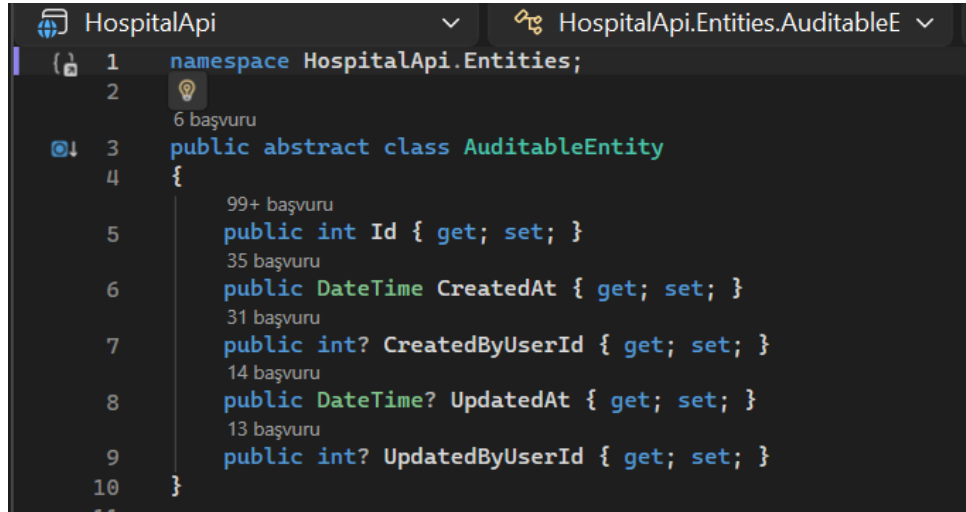
    await PageUi.WithSpinnerAsync(BusyOverlay, BusySpinner, async () =>
    {
        try
        {
            await _api.DeletePatientAsync(_patientId.Value);
            await Navigation.PopAsync();
        }
        catch (Exception ex)
        {
            await DisplayAlert("Sil", ex.Message, "Tamam");
        }
    });
}
```

İSTER 10 – Ekleme/Güncellemede Kim-Ne Zaman Bilgisi (Audit)

Tüm ana entity sınıfları (Patient, Doctor, Clinic, Appointment, LabReport, DoctorDuty) AuditableEntity isimli soyut temel sınıftan kalıtım alır. Bu temel sınıf; her kaydın kim tarafından, ne zaman oluşturulduğunu ve en son kim tarafından, ne zaman güncellendiğini otomatik olarak takip eden dört alan içerir.

Kayıt eklenirken CreatedAt ve CreatedByUserId alanları, işlemi gerçekleştiren kullanıcının kimliği ve o anki UTC zamanıyla doldurulur. Kayıt güncellenirken de UpdatedAt ve UpdatedByUserId alanları aynı şekilde güncellenir. Bu bilgiler sayesinde bir kaydı kimin oluşturduğu veya değiştirdiği sonradan sorgulanabilir; denetim (audit) gereklilikleri sağlanmış olur.

Kod: src/HospitalApi/Entities/AuditableEntity.cs



```
1 namespace HospitalApi.Entities;
2
3 public abstract class AuditableEntity
4 {
5     public int Id { get; set; }
6     public DateTime CreatedAt { get; set; }
7     public int? CreatedByUserId { get; set; }
8     public DateTime? UpdatedAt { get; set; }
9     public int? UpdatedByUserId { get; set; }
10 }
```

Kod: PatientService.cs – oluşturma ve güncelleme sırasında audit alanı ataması

```
85
86     var userId = currentUser.UserId;
87     var entity = new Patient
88     {
89         FirstName = request.FirstName.Trim(),
90         LastName = request.LastName.Trim(),
91         NationalId = request.NationalId.Trim(),
92         Phone = request.Phone.Trim(),
93         Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim(),
94         BirthDate = request.BirthDate.Date,
95         CreatedAt = DateTime.UtcNow,
96         CreatedByUserId = userId
97     };
98     entity.PortalPasswordHash = patientPasswordHasher.HashPassword(entity, "Patient123!");
99
100     db.Patients.Add(entity);
101     await db.SaveChangesAsync(cancellationToken);
102
103     var dto = await GetByIdAsync(entity.Id, cancellationToken);
104     return (true, null, dto);
105 }
106
107 2 başvuru
108 public async Task<(bool Success, string? Error)> UpdateAsync(int id, UpdatePatientRequest request, CancellationToken can
109 {
110     var entity = await db.Patients.FirstOrDefaultAsync(p => p.Id == id, cancellationToken);
111     if (entity is null)
112     {
113         return (false, "Hasta bulunamadı.");
114     }
115
116     var duplicate = await db.Patients.AnyAsync(
117         p => p.NationalId == request.NationalId && p.Id != id,
118         cancellationToken);
119     if (duplicate)
120     {
121         return (false, "Bu T.C. kimlik numarası başka bir hastaya ait.");
122     }
123
124     entity.FirstName = request.FirstName.Trim();
125     entity.LastName = request.LastName.Trim();
126     entity.NationalId = request.NationalId.Trim();
127     entity.Phone = request.Phone.Trim();
128     entity.Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim();
129     entity.BirthDate = request.BirthDate.Date;
130     entity.UpdatedAt = DateTime.UtcNow;
131     entity.UpdatedByUserId = currentUser.UserId;
132
133     await db.SaveChangesAsync(cancellationToken);
134     return (true, null);
135 }
```

İSTER 11 – Kullanıcı Girişi

Kullanıcı girişi JWT (JSON Web Token) tabanlı kimlik doğrulama ile sağlanmaktadır. Sistem üç farklı kullanıcı türünü destekler: Admin, Doktor ve Hasta. Her birinin giriş ekranı ve API endpoint'i ayrıdır; bu sayede roller birbirinden izole edilmiş olur.

Uygulama açıldığında portal seçim ekranı gösterilir. Kullanıcı hangi portala giriş yapmak istediğini seçer (Yönetim / Doktor / Hasta). Seçimden sonra ilgili giriş ekranına yönlendirilir. Giriş ekranında kullanıcı adı (veya T.C. kimlik) ve şifre girilir. API'ye POST isteği gönderilir; doğrulama başarılıysa bir JWT token döner ve istemci bu token'ı güvenli bellek içi depoya (IAuthTokenStore) kaydeder. Sonraki tüm API isteklerinde bu token Authorization başlığında taşınır.



Doktor portalı

Hoş geldiniz

Yalnızca kendi randevu listenize erişirsiniz; yönetim işlemleri kurumsal hesaptadır.

Bilgiler doğrulanamadı. T.C. no / kullanıcı adı veya şifreyi kontrol edin.

[Tekrar dene](#)

Portal kullanıcı adı

dr.ayilmaz

Şifre

..... [Göster](#)

☒ Kullanım koşullarını ve kişisel verilerin korunmasına ilişkin aydınlatmayı okudum, kabul ediyorum.

[Kullanıcı sözleşmesini oku](#)

[Güvenli giriş](#)

Şekil8: Doktor paneline yanlış giriş yapılması durumunda sisteme girişe izin verilmez.

Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs – login endpoint'leri

```
0 başvuru
public static class HospitalEndpoints
{
    1 başvuru
    public static void MapHospitalEndpoints(this WebApplication app)
    {
        app.MapPost("/api/auth/login", async (LoginRequest request, IAuthService authService) =>
        {
            var validation = RequestValidator.Validate(request);
            if (validation is not null)
            {
                return validation;
            }

            var result = await authService.LoginAsync(request);
            return result is null ? Results.Unauthorized() : Results.Ok(result);
        }).AllowAnonymous();

        app.MapPost("/api/auth/login/doctor", async (DoctorPortalLoginRequest request, IAuthService authService) =>
        {
            var validation = RequestValidator.Validate(request);
            if (validation is not null)
            {
                return validation;
            }

            var result = await authService.LoginDoctorPortalAsync(request);
            return result is null ? Results.Unauthorized() : Results.Ok(result);
        }).AllowAnonymous();
    }
}
```

Kod: src/HospitalApi/Services/AuthService.cs – JWT token üretimi

```
8 namespace HospitalApi.Security;
9
10 1.basvuru
11 public sealed class TokenService(IOptions<JwtOptions> options) : ITokenService
12 {
13     private readonly JwtOptions _options = options.Value;
14
15     2.basvuru
16     public string CreateStaffToken(AppUser user) =>
17         CreateToken(
18             user.Id.ToString(),
19             user.UserName,
20             user.FullName,
21             "Admin");
22
23     2.basvuru
24     public string CreateDoctorToken(Doctor doctor) =>
25         CreateToken(
26             doctor.Id.ToString(),
27             doctor.PortalUserName ?? string.Empty,
28             $"{doctor.FirstName} {doctor.LastName}",
29             "Doctor");
30
31     2.basvuru
32     public string CreatePatientToken(Patient patient) =>
33         CreateToken(
34             patient.Id.ToString(),
35             patient.NationalId,
36             $"{patient.FirstName} {patient.LastName}",
37             "Patient");
38
39     3.basvuru
40     private string CreateToken(string subjectId, string userName, string fullName, string role)
41     {
42         var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.SigningKey));
43         var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
44         var claims = new List<Claim>
45         {
46             new(ClaimTypes.NameIdentifier, subjectId),
47             new(ClaimTypes.Name, userName),
48             new(ClaimTypes.GivenName, fullName),
49             new(ClaimTypes.Role, role)
50         }
51     }
52 }
```

İSTER 12 – Kullanıcı Çıkışı

Oturum kapatma işlemi MAUI istemci tarafında gerçekleştirilir. JWT, stateless (durumsuz) bir kimlik doğrulama mekanizması olduğundan sunucu tarafında bir oturum kaydı tutulmaz. Çıkış yapmak için token'ın istemci tarafından silinmesi yeterlidir; token silindiğinde yetkilendirme başlığı gönderilmediği için sonraki API istekleri 401 Unauthorized ile reddedilir.

Her portal Shell'inin (Admin, Doktor, Hasta) alt menüsünde 'Çıkış Yap' butonu bulunur. Bu butona basıldığında IAuthTokenStore servisinin Clear() metodu çağrılarak token bellekten silinir; ardından kullanıcı portal seçim ekranına (LoginNavigationPage) yönlendirilir. Yönlendirme, Shell navigasyon yığını temizlenerek yapılır; böylece geri tuşuna basılarak oturuma geri dönülemez.



Şekil9: Bir portaldan çıkış

Kod: src/Hospital.Client/AppShell.xaml – FlyoutFooter çıkış butonu

```
<Shell.FlyoutFooter>
    <VerticalStackLayout Padding="16,12,16,14"
        Spacing="10"
        BackgroundColor="#E8F4FC">
        <views:HospitalFooterBar Margin="0,0,0,4" />
        <Label x:Name="FlyoutFooterVersion"
            FontSize="11"
            TextColor="{StaticResource TextSecondary}"
            HorizontalTextAlignment="Center" />
        <Label Text="Oturumu kapatınca tekrar giriş gerekir."
            FontSize="11"
            TextColor="{StaticResource TextSecondary}"
            HorizontalTextAlignment="Center"
            LineBreakMode="WordWrap" />
        <Button Text="Çıkış Yap"
            Style="{StaticResource OutlineButton}"
            Clicked="OnLogoutClicked" />
    </VerticalStackLayout>
</Shell.FlyoutFooter>
</Shell>
```

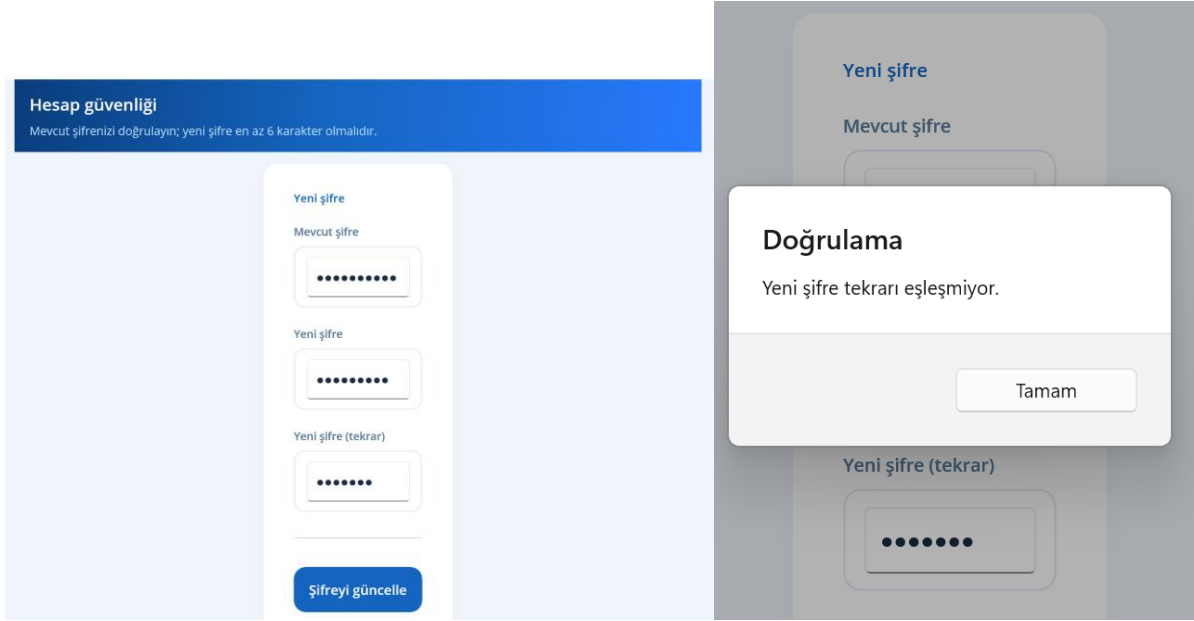
Kod: src/Hospital.Client/AppShell.xaml.cs – OnLogoutClicked

```
0 başvuru
private void OnLogoutClicked(object? sender, EventArgs e)
{
    var store = AppLocator.Services.GetRequiredService<IAuthTokenStore>();
    store.Clear();
    Application.Current!.Windows[0].Page = App.CreateLoginNavigation();
}
}
```

İSTER 13 – Şifre Değiştirme Sayfası

Sistemde şifre değiştirme işlemi için ayrı bir sayfa ve API endpoint'i mevcuttur. Admin panelinde sol flyout menüsünde 'Şifre Değiştir' seçeneği bulunur; bu seçeneğe tıklandığında ChangePasswordPage açılır.

Şifre değiştirme formunda üç alan bulunur: mevcut şifre, yeni şifre ve yeni şifre tekrarı. MAUI tarafında yeni şifre ile tekrarının eşleşip eşleşmediği kontrol edilir; eşleşmezse kullanıcıya hata gösterilir. İstek API'ye gönderildiğinde sunucu tarafında mevcut şifre doğrulanır; yanlışsa hata döner. Doğruysa yeni şifre hash'lenerek kaydedilir.



Şekil 10: Şifre değiştirme ekranı- Yanlış şifre girilmesi durumu

Kod: src/HospitalApi/Endpoints/HospitalEndpoints.cs – şifre değiştirme endpoint'i

```
app.MapPut("/api/auth/change-password", async (ChangePasswordRequest request, ClaimsPrincipal user)
{
    var validation = RequestValidator.Validate(request);
    if (validation is not null)
    {
        return validation;
    }

    var id = GetUserId(user);
    if (id is null)
    {
        return Results.Unauthorized();
    }

    var (success, error) = await authService.ChangePasswordAsync(id.Value, request);
    return success ? Results.NoContent() : Results.BadRequest(new { error });
}).RequireAuthorization("Admin");
```

Kod: src/HospitalApi/Services/AuthService.cs – ChangePasswordAsync

```
2 başvuru
public async Task<(bool Success, string? Error)> ChangePasswordAsync(int userId, ChangePasswordRequest request)
{
    var user = await db.Users.FirstOrDefaultAsync(u => u.Id == userId, cancellationToken);
    if (user is null)
    {
        return (false, "Kullanıcı bulunamadı.");
    }

    var verification = passwordHasher.VerifyHashedPassword(user, user.PasswordHash, request.CurrentPassword);
    if (verification == PasswordVerificationResult.Failed)
    {
        return (false, "Mevcut şifre hatalı.");
    }

    user.PasswordHash = passwordHasher.HashPassword(user, request.NewPassword);
    await db.SaveChangesAsync(cancellationToken);
    return (true, null);
}
```

İSTER 14 – CollectionView Kullanımı

MAUI'nin CollectionView kontrolü, büyük veri kümelerini yüksek performansla ve özelleştirilebilir şablonlarla listelemek için kullanılır. Geleneksel ListView'a kıyasla CollectionView; çift yön kaydırma, seçim modu, boş durum görünümü ve özel düzen desteği gibi gelişmiş özellikler sunar.

Projede CollectionView; hasta, doktor, klinik, randevu, lab raporu, doktor görevi ve hasta zaman çizelgesi gibi tüm liste ekranlarında kullanılmaktadır. Her ekranda özel bir DataTemplate tanımlanmış, kartlar Border kontrolü ile yuvarlak köşeli tasarım kazanmıştır. CollectionView, RefreshView içine sarılmıştır; böylece 'aşağı çek-yenile' (pull-to-refresh) özelliği ücretsiz kazanılmıştır. SelectionMode='Single' ile tıklama olayı SelectionChanged event'i üzerinden yönetilir.

Doktor kadrosu

İsim, uzmanlık veya poliklinik ile filtreleyin; kayıt seçerek düzenleyin.

+ Yeni doktor kaydı oluştur

Arama ve filtre

Poliklinik filtresi

101 — Dahiliye

Kağan Yıldız

Dahiliye

Dahiliye

Ayşe Yılmaz

Dahiliye

Dahiliye

Şekil11: Dahiliye doktorlarının listelenmesi durumu

Kod: src/Hospital.Client/PatientsPage.xaml – CollectionView + RefreshView

```
Grid Grid.Row="1">
    RefreshView x:Name="RefreshHost" Refreshing="OnRefreshing">
        <CollectionView x:Name="PatientsCollection"
            Style="{StaticResource AppListCollectionView}"
            SelectionMode="Single"
            SelectionChanged="OnSelectionChanged"
            SemanticProperties.Description="Hasta listesi; bir satıra dokunarak detayı açın.">
            <CollectionView.EmptyView>
                <views:EmptyStateView Title="Henüz hasta kaydı yok"
                    Message="Arama kriterlerinizi değiştirin veya yeni hasta oluşturmak için ara"
                    Hint="Sunucu bağlantısı yoksa API adresini kontrol edin." />
            />
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Border Style="{StaticResource ListRowCard}">
                        <Grid ColumnDefinitions="*,Auto" RowSpacing="4">
                            <VerticalStackLayout Spacing="6" HorizontalOptions="Fill">
                                <HorizontalStackLayout Spacing="8">
                                    <Label Text="{Binding FirstName}" FontFamily="OpenSansSemibold" FontSize="17"
                                        <Label Text="{Binding LastName}" FontFamily="OpenSansSemibold" FontSize="17" /
                                </HorizontalStackLayout>
                                <Label Text="{Binding NationalId}" FontSize="13" TextColor="{StaticResource Text"
                                    <Label Text="{Binding Phone}" FontSize="13" TextColor="{StaticResource TextSeco
                                </VerticalStackLayout>
                                <Label Grid.Column="1"
                                    Text="{Binding BirthDate, StringFormat='{0:d}'}"
                                    FontSize="13"
                                    FontFamily="OpenSansSemibold"
                                    TextColor="{StaticResource Primary}"
                                    VerticalTextAlignment="Center" />
                                </Grid>
                            </Border>
                        </DataTemplate>
                    </CollectionView.ItemTemplate>
                <CollectionView.Footer>
                    <Label Padding="0,12,0,4"
                        FontSize="13"
                    />
                </CollectionView.Footer>
            </CollectionView>
        </RefreshView>
    </Grid>
```

Kod: src/Hospital.Client/PatientsPage.xaml.cs – seçim olayı

```
0 başvuru
private async void OnSelectionChanged(object? sender, SelectionChangedEventArgs e)
{
    if (DutiesCollection.SelectedItem is not DoctorDutyListRow d)
    {
        return;
    }

    DutiesCollection.SelectedItem = null;
    await Navigation.PushAsync(new DoctorDutyEditPage(d.Source.Id));
}
```

İSTER 15 – Picker Kullanımı

MAUI Picker kontrolü, önceden tanımlı seçenekler listesinden bir öğe seçmek için kullanılır. Projede Picker; randevu oluştururken klinik ve doktor seçiminde, randevu ve lab raporu listelerini filtrelerken, hasta zaman çizelgesinde kullanılmaktadır.

Randevu oluşturma akışında Picker kullanımı özellikle dikkat çekicidir: Önce klinik Picker'ından bir klinik seçilir; seçim gerçekleştiğinde SelectedIndexChanged event'i tetiklenir ve seçilen kliniğe ait doktorlar API'den yüklenerek ikinci Picker'a (DoctorPicker) aktarılır. Böylece doktor listesi, kliniğe bağlı dinamik olarak güncellenir. Klinik seçilmeden önce DoctorPicker devre dışı (IsEnabled=false) kalır. Aynı sayfada tarih seçimi için DatePicker, saat seçimi için zaman slot listesi de kullanılmaktadır.

The image shows two side-by-side screenshots of a mobile application interface. The left screenshot, titled 'Doktor kadrosu', features a blue header with the text 'İsim, uzmanlık veya poliklinik ile filtreleyin; kayıt seçerek düzenleyin.' Below the header is a green button labeled '+ Yeni doktor kaydı oluştur'. Underneath is a search bar labeled 'Arama ve filtre' with the placeholder text 'Ad, uzmanlık veya poliklinik'. Below the search bar is a dropdown menu labeled 'Poliklinik filtresi' with the selected option '102 — Kardiyoloji'. At the bottom, there are two doctor profiles: 'Mehmet Kaya' (Kardiyolog, Kardiyoloji) and 'Burak Tunç' (Kardiyolog, Kardiyoloji). The right screenshot, titled 'Doktor kaydı', has a blue header with the text 'Ad, uzmanlık ve bağlı olduğu poliklinik bilgilerini düzenleyin.' Below the header is a form with fields for 'Ad' (Beril), 'Soyad' (Tetik), 'Uzmanlık / branş' (Uzman), and 'Poliklinik' (Poliklinik seçin, Kardiyoloji). A blue 'Kaydet' button is at the bottom.

Şekil12: Görselde Kardiyoloji bölümüne yeni doktor kaydı oluşturma kısmı gösterilmiştir.

Kod: src/Hospital.Client/PatientBookAppointmentPage.xaml – Picker çifti

```
<Grid Grid.Row="1">
  <ScrollView BackgroundColor="Transparent">
    <Border Style="{StaticResource FormSheetCard}" Margin="{StaticResource PageFormSheetMargin}" Paddi
      <VerticalStackLayout Spacing="18">
        <VerticalStackLayout Spacing="8">
          <Label Text="Poliklinik" Style="{StaticResource SectionLabel}" />
          <Border Style="{StaticResource FormFieldBorder}" Padding="6,2">
            <Picker x:Name="ClinicPicker"
              Title="Dizin seçin"
              TextColor="{StaticResource TextPrimary}" />
          </Border>
        </VerticalStackLayout>
        <VerticalStackLayout Spacing="8">
          <Label Text="Doktor" Style="{StaticResource SectionLabel}" />
          <Border Style="{StaticResource FormFieldBorder}" Padding="6,2">
            <Picker x:Name="DoctorPicker"
              Title="Önce poliklinik seçin"
              TextColor="{StaticResource TextPrimary}" />
          </Border>
        </VerticalStackLayout>
      </Border>
    </ScrollView>
  </Grid>
```

Kod: src/Hospital.Client/AppointmentsPage.xaml – liste filtresi Picker'ları

```
      TextColor="{StaticResource TextPrimary}" />
    </Border>
    <Border Style="{StaticResource FormFieldBorder}" Padding="6,2">
      <Picker x:Name="ClinicFilterPicker"
        Title="Poliklinik filtresi"
        TextColor="{StaticResource TextPrimary}" />
    </Border>
  </VerticalStackLayout>
</Border>
```

İSTER 16 – DatePicker (veya TimePicker) Kullanımı

MAUI DatePicker kontrolü, kullanıcıdan tarih bilgisi almak için platforma özel tarih seçici (native date picker) sunar. Bu kontrol; hasta doğum tarihi, randevu tarihi ve lab raporu tarihlerinin girilmesinde kullanılmaktadır.

Hasta kayıt/düzenleme formunda (PatientEditPage) bir DatePicker ile hasta doğum tarihi seçilir. Randevu oluşturma sayfasında ise randevu tarihi DatePicker ile seçildikten sonra ilgili güne ait müsait zaman slotları API'den yüklenerek liste hâlinde gösterilir. Lab raporu formunda (LabReportEditPage) da tetkik sonuç tarihi DatePicker ile girilir. Bu kullanım; ister 16'nın 'DatePicker veya TimePicker kullanılacak' şartını birden fazla sayfada karşılamaktadır.

The image displays two side-by-side screenshots of a medical application interface. The left screenshot, titled 'Hasta kaydı' (Patient Registration), shows a form with fields for 'Ad' (Name) and 'Soyad' (Surname). A date picker is open, showing the month of Haziran 2000 (June 2000) with the 15th day selected. The right screenshot, titled 'Randevu planı' (Appointment Scheduling), shows a form with fields for 'Doktor seçin' (Select Doctor), 'Zamanlama' (Scheduling), 'Tarih' (Date), 'Durum' (Status), and 'Notlar' (Notes). A time picker is open, showing a list of times from 08:30 to 11:30, with 10:00 selected.

Şekil 13: Tarih ve saat seçimi

Kod: src/Hospital.Client/PatientEditPage.xaml – doğum tarihi DatePicker

```
        </Border>
    </VerticalStackLayout>
</Grid>

<VerticalStackLayout Spacing="8">
    <Label Text="Doğum tarihi" Style="{StaticResource SectionLabel}" Margin="0" />
    <Border Style="{StaticResource FormFieldBorder}" Padding="10,4">
        <DatePicker x:Name="BirthDatePicker"
                    HorizontalOptions="Start"
                    TextColor="{StaticResource TextPrimary}" />
    </Border>
</VerticalStackLayout>

<BoxView HeightRequest="1" Color="{StaticResource BorderSubtle}" Margin="0,8,0,4" />

<Button Text="Kaydet"
        Margin="0,4,0,0"
        Clicked="OnSaveClicked" />
<Button x:Name="DeleteButton"
        Text="Kaydı sil"
        Style="{StaticResource DangerButton}"
        Clicked="OnDeleteClicked"
        IsVisible="False" />
</VerticalStackLayout>
```

İSTER 17 – CheckBox ve RadioButton Kullanımı

Projede hem CheckBox hem de RadioButton kontrolleri aktif olarak kullanılmaktadır. İster 17, 'CheckBox veya RadioButton' şartını talep etmekte; ancak projede her ikisi de mevcuttur.

CheckBox kullanımı: Admin ve doktor giriş ekranlarında 'Kullanım koşullarını okudum ve kabul ediyorum' onay kutusu (CheckBox) bulunur. Kullanıcı bu onay kutusunu işaretlemeden 'Giriş Yap' butonu devre dışı kalır (IsEnabled'ı CheckBox'ın IsChecked değerine bağlıdır). Bu yapı; gizlilik politikası veya kullanım şartlarını kullanıcının açıkça onaylamasını zorunlu kılmak için gerçekçi bir senaryo oluşturmaktadır.

RadioButton kullanımı: Randevular sayfasında tarih filtresi için üç adet RadioButton bulunur: 'Tüm tarihler', 'Bugün' ve 'Bu hafta'. Bu RadioButton'lar aynı GroupName grubuna ait olduğundan aynı anda yalnızca biri seçili olabilir. Seçim değiştiğinde CheckedChanged event'i tetiklenerek filtre otomatik uygulanır; kullanıcının ayrıca 'Filtrele' butonuna basması gerekmez.

Kullanıcı adı

Şifre

Göster

☒ Kullanım koşullarını ve kişisel verilerin korunmasına ilişkin aydınlatmayı okudum, kabul ediyorum.
[Kullanıcı sözleşmesini oku](#)

Güvenli giriş

Kullanıcı Sözleşmesi

HASTANE BİLGİ SİSTEMİ KULLANICI SÖZLEŞMESİ

- Amaç**
Bu sistem, hastane hizmetlerinin elektronik ortamda yürütülmesi amacıyla geliştirilmiştir. Kullanıcılar sisteme erişerek aşağıdaki koşulları kabul etmiş sayılır.
- Kişisel Verilerin Korunması**
Sisteme girilen tüm kişisel veriler 6698 sayılı KVKK kapsamında işlenir. Verileriniz yalnızca sağlık hizmeti sunumu ve yasal yükümlülükler çerçevesinde kullanılır; üçüncü taraflarla paylaşılmaz.
- Hesap Güvenliği**
Kullanıcı adı ve şifrenizin gizliliğinden siz sorumlusunuz. Hesabınızla gerçekleştirilen tüm işlemler size ait kabul edilir. Şüpheli bir durumda derhal bilgi işlem birimine başvurunuz.
- Kullanım Kuralları**
Sistemi yalnızca yetkilendirilmiş işlemler için kullanabilirsiniz. Yetkisiz erişim girişimi, verilerin kopyalanması veya manipülasyonu yasaktır.
- Sorumluluk**
Kurum, sistemin kesintisiz çalışacağını garanti etmez. Teknik bakım ve güncelleme dönemlerinde erişim geçici olarak kısıtlanabilir.
- Değişiklikler**
Kurum bu sözleşmeyi önceden bildirimde bulunmaksızın güncelleme hakkını saklı tutar. Güncel metin her zaman giriş ekranında erişilebilir durumdadır.

Tamam

Şekil14:Kullanım koşulları onaylama kısmı, sözleşme oku butonuna tıklandığında ekrana sözleşme gelir.

Randevu operasyon ekranı

Tarih aralığı ve durum seçin; KPI'lar seçili listeyi yansıtır. Satır seçerek düzenleyin.

+ Yeni randevu oluştur

0

Toplam

0

Planlı

0

Tamamlanan

0

İptal

Tarih görünümü

☒ Tüm tarihler

☐ Bugün

☐ Bu hafta

Durum filtresi

Randevu durumu

Tüm durumlar

Planlı

Tamamlandı

İptal

Randevu: +90 (246) 211 00 00 - Acil: 112 - E-posta: iletisim@isubuhastane.tr

Şekil15: Randevu planında Tüm tarihler-Bugün-Bu hafta RadioButton kullanımı

Kod: src/Hospital.Client/LoginPage.xaml – CheckBox ile koşul kabulü

```
</Grid>

<Button Text="Güvenli giriş"
        BackgroundColor="{StaticResource Primary}"
        TextColor="White"
        FontFamily="OpenSansSemibold"
        CornerRadius="10"
        Padding="18,14"
        SemanticProperties.Hint="Sunucuya bağlanarak oturum açar"
        Clicked="OnLoginClicked" />
```

```
</VerticalStackLayout>

<Grid ColumnDefinitions="Auto,*"
      ColumnSpacing="10"
      RowSpacing="0">
    <CheckBox x:Name="AcceptTermsCheckBox"
              Grid.Column="0"
              VerticalOptions="Start"
              Margin="0,2,0,0" />
    <VerticalStackLayout Grid.Column="1" Spacing="2" VerticalOptions="Center">
        <Label Text="Kullanım koşullarını ve kişisel verilerin korunmasına :
                FontSize="13"
                TextColor="{StaticResource TextPrimary}"
                LineBreakMode="WordWrap" />
        <Label Text="Kullanıcı sözleşmesini oku"
                FontSize="12"
                FontFamily="OpenSansSemibold" />
    </VerticalStackLayout>
</Grid>
```

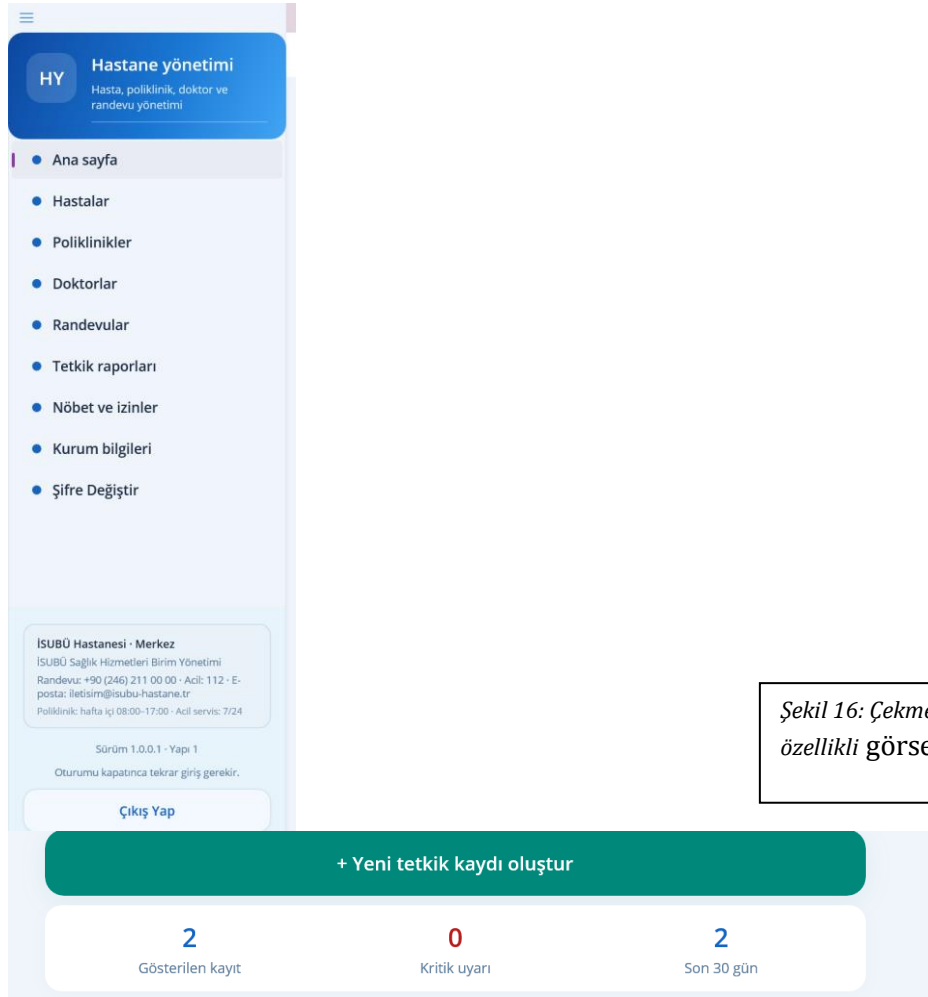
Kod: src/Hospital.Client/AppointmentsPage.xaml – RadioButton tarih filtresi

```
<VerticalStackLayout Spacing="14">
    <Label Text="Tarih görünümü"
          Style="{StaticResource FormSubSectionTitle}"
          Margin="0,-4,0,-4" />
    <HorizontalStackLayout Spacing="10">
        <RadioButton x:Name="RbAll" Content="Tüm tarihler" GroupName="ApptRange" />
        <RadioButton x:Name="RbToday" Content="Bugün" GroupName="ApptRange" />
        <RadioButton x:Name="RbWeek" Content="Bu hafta" GroupName="ApptRange" />
    </HorizontalStackLayout>
    <BoxView HeightRequest="1" Color="{StaticResource BorderSubtle}" Margin="0,4,0,4" />
    <Label Text="Durum filtresi">
```

İSTER 18 – Derste Anlatılmayan MAUI Kontrolleri

Temel Entry/Label/Button/Image kontrollerinin ötesinde çeşitli ileri düzey MAUI bileşenleri kullanılmıştır. Bu bileşenler; uygulamanın kullanılabilirliğini artırmak ve modern bir kullanıcı deneyimi sunmak amacıyla tercih edilmiştir.

- **Shell:** Shell navigasyon mimarisini (FlyoutItem, ShellContent, FlyoutHeader, FlyoutFooter) destekleyen, menü çekmeceli (hamburger) düzeni sağlayan yapı
- **RefreshView:** Kullanıcının listeyi aşağı çekerek içeriği yenilemesini sağlayan, CollectionView'ı saran kontrol
- **Border:** Yuvarlak köşeli kartlar oluşturmak için StrokeShape özelliği ile kullanılan kaplama kontrolü
- **RoundRectangle:** Görsellerin dairesel veya yuvarlak köşeli gösteriminde kullanılan şekil sınıfı
- **Switch:** İki değer arasında açma/kapama kontrolü (örn: doktor izin aktif/pasif durumu)
- **HandHoverBehavior (Custom Behavior):** Fareyle üzerine gelindiğinde imleç tipini değiştiren özel davranış sınıfı
- **Editor (çok satırlı metin):** Giriş alanlarında klavye tipini ayarlamak için kullanılan, Entry kontrolünün genişletilmiş hali



Şekil 16: Çekmeceli menu ve StrokeShape özellikli görseller

+ Yeni poliklinik ekle

Arama

İsim veya açıklama

G

Görme alanı, OCT ve fundus görüntüleme ile takip; cerrahi öncesi/sonrası danışmanlık bu birim koordinasyonunda yürütülür.

Doktorları gör Düzenle



102 — Kardiyoloji

Kalp ve damar hastalıklarında risk değerlendirmesi, EKO, efor testi ve Holter istemi. Koroner arter hastalığı, aritmi ve kalp yetmezliğinde tedavi planı oluşturulur; ameliyat öncesi kardiyolojik uygunluk raporları hazırlanır.

Doktorları gör Düzenle

K

105 — KBB

İşitme denge ve sinüs hastalıkları, bademcik ve geniz eti değerlendirmesi, ses teli muayeneleri. Endoskopik muayene ve işitme testleri için randevu planlanır; acil KBB durumlarında yönlendirme protokolü uygulanır.

Doktorları gör Düzenle

N

106 — Nöroloji

Baş ağrısı, baş dönmesi, inme riski, epilepsi ve hafıza şikayetlerinde nörolojik muayene ve

Şekil17: Border + RoundedRectangle ile oluşturulmuş klinik kartları; yuvarlak köşeli kartlar listesi

Kod: AppShell.xaml – Shell + FlyoutHeader + FlyoutFooter

```
<FlyoutItem Title="Ana sayfa">
|   <ShellContent ContentTemplate="{DataTemplate local:AdminHomePage}" Route="AdminHomePage" />
| </FlyoutItem>

<FlyoutItem Title="Hastalar">
|   <ShellContent ContentTemplate="{DataTemplate local:PatientsPage}" Route="PatientsPage" />
| </FlyoutItem>

<FlyoutItem Title="Poliklinikler">
|   <ShellContent ContentTemplate="{DataTemplate local:ClinicsPage}" Route="ClinicsPage" />
| </FlyoutItem>

<FlyoutItem Title="Doktorlar">
|   <ShellContent ContentTemplate="{DataTemplate local:DoctorsPage}" Route="DoctorsPage" />
| </FlyoutItem>

<FlyoutItem Title="Randevular">
|   <ShellContent ContentTemplate="{DataTemplate local:AppointmentsPage}" Route="AppointmentsPage" />
| </FlyoutItem>

<FlyoutItem Title="Tetkik raporları">
|   <ShellContent ContentTemplate="{DataTemplate local:AdminLabReportsPage}" Route="AdminLabReportsPage" />
| </FlyoutItem>
```

Kod: PatientsPage.xaml – RefreshView + Border kart

```
02         </Border>
03         <Grid Grid.Row="11">
04             <RefreshView x:Name="RefreshHost" Refreshing="OnRefreshing">
05                 <CollectionView x:Name="PatientsCollection"
06                     Style="{StaticResource ApplistCollectionView}"
07                     SelectionMode="Single"
08                     SelectionChanged="OnSelectionChanged"
09                     SemanticProperties.Description="Hasta listesi; bir satıra dokunarak detayı açın.">
10                     <CollectionView.EmptyView>
11                         <views:EmptyStateView Title="Henüz hasta kaydı yok"
12                             Message="Arama kriterlerinizi değiştirin veya yeni hasta oluşturmak için"
13                             Hint="Sunucu bağlantısı yoksa API adresini kontrol edin." />
14                     </CollectionView.EmptyView>
15                     <CollectionView.ItemTemplate>
16                         <DataTemplate>
17                             <Border Style="{StaticResource ListRowCard}">
18                                 <Grid ColumnDefinitions="*,Auto" RowSpacing="4">
19                                     <VerticalStackLayout Spacing="6" HorizontalOptions="Fill">
20                                         <HorizontalStackLayout Spacing="8">
21                                             <Label Text="{Binding FirstName}" FontFamily="OpenSansSemibold" FontSize="12"
22                                                 <Label Text="{Binding LastName}" FontFamily="OpenSansSemibold" FontSize="12"
23                                         </HorizontalStackLayout>
24                                         <Label Text="{Binding NationalId}" FontSize="13" TextColor="{StaticResource TextSecondary}"
25                                         <Label Text="{Binding Phone}" FontSize="13" TextColor="{StaticResource TextPrimary}"
26                                     </VerticalStackLayout>
27                                     <Label Grid.Column="1"
28                                         Text="{Binding BirthDate, StringFormat='{0:d}'}"
29                                         FontSize="13"
30                                         FontFamily="OpenSansSemibold"
31                                         TextColor="{StaticResource TextPrimary}"
32                                         VerticalTextAlignment="Center" />
33                                 </Grid>
34                             </Border>
35                         </DataTemplate>
36                     </CollectionView.ItemTemplate>
37                     <CollectionView.Footer>
38                         <Label Padding="0,12,0,4"
39                             FontSize="11"
40                             TextColor="{StaticResource TextSecondary}"
41                             HorizontalTextAlignment="Center"
42                             Text="Kayıtlar sunucudan yüklenir. Bağlantı sorununda API adresini kontrol edin." />
43                     </CollectionView.Footer>
44                 </CollectionView>
45             </RefreshView>
46             <Grid x:Name="BusyOverlay"
47                 IsVisible="False"
48                 BackgroundColor="#DDF5FAFF"
49                 ZIndex="1000">
50             </Grid>
51         </Grid>
52     </Content>
53 </Page>
```

İSTER 19 – Service Interface'leri

Proje, Dependency Inversion Principle (DIP) doğrultusunda service interface pattern kullanmaktadır. Her iş servisi; somut implementasyon sınıfından önce bir interface ile tanımlanmıştır. API katmanı servis implementasyonlarına doğrudan değil, interface'leri üzerinden bağımlıdır. Bu sayede testlerde mock implementasyonlar kolayca enjekte edilebilir ve sınıflar birbirinden gevşek bağımlı (loosely coupled) kalır.

API tarafında IPatientService, IAuthService, IClinicService, IDoctorService, IAppointmentService, ILabReportService, IDoctorDutyService, IPortalAppointmentService ve IPortalTimelineService interface'leri tanımlanmıştır. Client tarafında ise IHospitalApiClient ve IAuthTokenStore interface'leri mevcuttur.

Kod: src/HospitalApi/Services/IPatientService.cs

```
1 using Hospital.Shared.Dtos;
2
3 namespace HospitalApi.Services;
4
5 10 başvuru
6 public interface IPatientService
7 {
8     3 başvuru
9     Task<IReadOnlyList<PatientDto>> GetAsync(PatientListQuery query, CancellationToken cancellationToken = default);
10    5 başvuru
11    Task<PatientDto?> GetByIdAsync(int id, CancellationToken cancellationToken = default);
12    2 başvuru
13    Task<(bool Success, string? Error, PatientDto? Patient)> CreateAsync(CreatePatientRequest request, CancellationToken cancellationToken = default);
14    2 başvuru
15    Task<(bool Success, string? Error)> UpdateAsync(int id, UpdatePatientRequest request, CancellationToken cancellationToken = default);
16    2 başvuru
17    Task<(bool Success, string? Error)> DeleteAsync(int id, CancellationToken cancellationToken = default);
18
19    2 başvuru
20    Task<(bool Success, string? Error, PatientDto? Patient)> UpdatePortalProfileAsync(
21        int patientId,
22        UpdatePatientProfileRequest request,
23        CancellationToken cancellationToken = default);
24 }
```

Kod: src/HospitalApi/Program.cs – tüm servis interface DI kayıtları

```
builder.Services.AddSingleton<IPasswordHasher<AppUser>, PasswordHasher<AppUser>>();
builder.Services.AddSingleton<IPasswordHasher<Doctor>, PasswordHasher<Doctor>>();
builder.Services.AddSingleton<IPasswordHasher<Patient>, PasswordHasher<Patient>>();
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserContext, CurrentUserContext>();
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));
builder.Services.AddScoped<ITokenService, TokenService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IPatientService, class HospitalApi.Security.TokenService>();
builder.Services.AddScoped<IClinicService, ClinicService>();
builder.Services.AddScoped<IDoctorService, DoctorService>();
builder.Services.AddScoped<IAppointmentService, AppointmentService>();
builder.Services.AddScoped<ILabReportService, LabReportService>();
builder.Services.AddScoped<IDoctorDutyService, DoctorDutyService>();
builder.Services.AddScoped<IPortalAppointmentService, PortalAppointmentService>();
builder.Services.AddSingleton<ILabPdfStorage, LabPdfStorage>();
builder.Services.AddScoped<IPortalTimelineService, PortalTimelineService>();
```

İSTER 20 – Service Sınıfları

Her interface'in karşılığında somut bir servis sınıfı bulunmaktadır. Bu sınıflar; constructor injection ile HospitalDbContext, ICurrentUserContext ve gerekli yardımcı servisleri alır. Servis sınıflarının tamamı sealed olarak işaretlenmiştir; bu tasarım kararı, kasıtsız kalıtımın önüne geçer ve derleyicinin optimizasyon yapmasına yardımcı olur.

PatientService sınıfı örnek alındığında; GetAsync metodu filtreleme ve sıralama, CreateAsync metodu yeni hasta kaydı, UpdateAsync metodu güncelleme ve DeleteAsync metodu silme işlemini gerçekleştirir. Her metot asenkron (async/await) çalışır ve CancellationToken parametresi ile iptal desteği sağlar.

Kod: src/HospitalApi/Services/PatientService.cs (sınıf başlığı ve constructor)

```
using Hospital.Shared.Dtos;

namespace HospitalApi.Services;

10 başvuru
public interface IPatientService
{
    3 başvuru
    Task<IReadOnlyList<PatientDto>> GetAsync(PatientListQuery query, CancellationToken cancellationToken = default);
    5 başvuru
    Task<PatientDto> GetByIdAsync(int id, CancellationToken cancellationToken = default);
    2 başvuru
    Task<(bool Success, string? Error, PatientDto? Patient)> CreateAsync(CreatePatientRequest request, CancellationToken cancellationToken = default);
    2 başvuru
    Task<(bool Success, string? Error)> UpdateAsync(int id, UpdatePatientRequest request, CancellationToken cancellationToken = default);
    2 başvuru
    Task<(bool Success, string? Error)> DeleteAsync(int id, CancellationToken cancellationToken = default);

    2 başvuru
    Task<(bool Success, string? Error, PatientDto? Patient)> UpdatePortalProfileAsync(
        int patientId,
        UpdatePatientProfileRequest request,
        CancellationToken cancellationToken = default);
}
```

Servis Sınıfları Listesi

```
builder.Services.AddSingleton<IPasswordHasher<AppUser>>, PasswordHasher<AppUser>>();
builder.Services.AddSingleton<IPasswordHasher<Doctor>>, PasswordHasher<Doctor>>();
builder.Services.AddSingleton<IPasswordHasher<Patient>>, PasswordHasher<Patient>>();
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserContext, CurrentUserContext>();
builder.Services.Configure<JwtOptions>(builder.Configuration.GetSection("Jwt"));
builder.Services.AddScoped<ITokenService, TokenService>();
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IPatientService, PatientService>();
builder.Services.AddScoped<IClinicService, ClinicService>();
builder.Services.AddScoped<IDoctorService, DoctorService>();
builder.Services.AddScoped<IDoctorDutyService, DoctorDutyService>();
builder.Services.AddScoped<IAppointmentService, AppointmentService>();
builder.Services.AddScoped<ILabReportService, LabReportService>();
builder.Services.AddScoped<IDoctorDutyService, DoctorDutyService>();
builder.Services.AddScoped<IPortalAppointmentService, PortalAppointmentService>();
builder.Services.AddSingleton<ILabPdfStorage, LabPdfStorage>();
builder.Services.AddScoped<IPortalTimelineService, PortalTimelineService>();
```

- Services
 - C# AppointmentService.cs
 - C# AuthService.cs
 - C# ClinicService.cs
 - C# DoctorDutyService.cs
 - C# DoctorService.cs
 - C# IAppointmentService.cs
 - C# IAuthService.cs
 - C# IClinicService.cs
 - C# IDoctorDutyService.cs
 - C# IDoctorService.cs
 - C# ILabReportService.cs
 - C# IPatientService.cs
 - C# IPortalAppointmentService.cs
 - C# IPortalTimelineService.cs
 - C# LabPdfStorage.cs
 - C# LabReportService.cs
 - C# PatientBookingHelper.cs
 - C# PatientService.cs
 - C# PortalAppointmentService.cs
 - C# PortalTimelineService.cs

İSTER 21 – OOP (Nesne Yönelimli Programlama) Prensipleri

Proje, nesne yönelimli programlamanın dört temel prensibini — soyutlama, kapsülleme, kalıtım ve çok biçimlilik — sistematik biçimde uygular.

Soyutlama (Abstraction)

Interface'ler ve abstract sınıflar sistemi soyutlamak için kullanılır. IPatientService interface'i, hasta yönetimi için gereken operasyonları tanımlar; ancak nasıl gerçekleştirildiğini söylemez. AuditableEntity abstract sınıfı, tüm kayıt türleri için ortak alanları (Id, CreatedAt, vb.) soyut bir sözleşme olarak tanımlar.

```
1 namespace HospitalApi.Entities;
2
3 6 başvuru
4 public abstract class AuditableEntity
5 {
6     99+ başvuru
7     public int Id { get; set; }
8     35 başvuru
9     public DateTime CreatedAt { get; set; }
10    31 başvuru
11    public int? CreatedByUserId { get; set; }
12    14 başvuru
13    public DateTime? UpdatedAt { get; set; }
14    13 başvuru
15    public int? UpdatedByUserId { get; set; }
16 }
```

Kapsülleme (Encapsulation)

Servis sınıfları sealed olarak işaretlenmiş, iç state private constructor injection ile saklanmıştır. Entity'lerin alanları property ile erişime açılmıştır. Şifre hash'leme mantığı servis katmanına kapsüllenmiştir; dışarıdan ham şifre görünmez.

```
0 public sealed class PatientService(HospitalDbContext db, ICurrentUserContext currentUser, IPasswordHasher<Patient>
1 {
2     3 başvuru
3     public async Task<IReadOnlyList<PatientDto>> GetAsync(PatientListQuery query, CancellationToken cancellationT
4     {
5         var linq = db.Patients.AsNoTracking().AsQueryable();
6
7         if (!string.IsNullOrEmpty(query.Search))
8         {
9             var term = query.Search.Trim();
10            linq = linq.Where(p =>
11                p.FirstName.Contains(term) ||
12                p.LastName.Contains(term) ||
13                (p.Email != null && p.Email.Contains(term)) ||
14                p.Phone.Contains(term));
15        }
16    }
```

Kalıtım (Inheritance)

Patient, Doctor, Clinic, Appointment, LabReport ve DoctorDuty sınıflarının tamamı AuditableEntity abstract sınıfından türer. Bu sayede audit alanları her sınıfta tekrar yazılmaz ve tüm entity'ler tutarlı bir id/zaman/kullanıcı yapısına sahip olur.

```
1 namespace HospitalApi.Entities;
2
3 public sealed class Patient : AuditableEntity
4 {
5     17 başvuru
6     public string FirstName { get; set; } = string.Empty;
7     17 başvuru
8     public string LastName { get; set; } = string.Empty;
9     23 başvuru
10    public string NationalId { get; set; } = string.Empty;
11    12 başvuru
12    public string Phone { get; set; } = string.Empty;
13    8 başvuru
14    public string? Email { get; set; }
15    9 başvuru
16    public DateTime BirthDate { get; set; }
17
18    4 başvuru
19    public string? EmergencyContactName { get; set; }
20    4 başvuru
21    public string? EmergencyContactPhone { get; set; }
22    4 başvuru
23    public string? EmergencyContactRelation { get; set; }
24
25    /// <summary>Hasta portal şifresi; boşsa hasta girişi kapalıdır.</summary>
26    16 başvuru
27    public string? PortalPasswordHash { get; set; }
28
29    3 başvuru
30    public ICollection<Appointment> Appointments { get; set; } = new List<Appointment>();
31
32    1 başvuru
33    public ICollection<LabReport> LabReports { get; set; } = new List<LabReport>();
34 }
```

Çok Biçimlilik (Polymorphism)

DI konteyneri, interface türüne göre doğru servis sınıfını otomatik enjekte eder. Endpoint'ler IPatientService interface'i üzerinden çalışır; somut sınıfı bilmeleri gerekmez. Bu sayede ileride PatientService yerine başka bir implementasyon konulabilir ve endpoint kodunun tek satırı değişmez.

```
builder.Services.AddScoped<IAuthService, AuthService>();
builder.Services.AddScoped<IPatientService, PatientService>();
builder.Services.AddScoped<IClinicService, ClinicService>();
builder.Services.AddScoped<IDoctorService, DoctorService>();
builder.Services.AddScoped<IAppointmentService, AppointmentService>();
builder.Services.AddScoped<ILabReportService, LabReportService>();
builder.Services.AddScoped<IDoctorDutyService, DoctorDutyService>();
builder.Services.AddScoped<IPortalAppointmentService, PortalAppointmentService>();
builder.Services.AddSingleton<ILabPdfStorage, LabPdfStorage>();
builder.Services.AddScoped<IPortalTimelineService, PortalTimelineService>();
```

İSTER 22 – Form Doğrulama (Validation)

Veri doğrulama, projede üç ayrı katmanda uygulanmaktadır. Bu yaklaşım; hataların mümkün olduğunca erken yakalanmasını ve her katmanda anlamlı geri bildirim sağlanmasını garanti eder.

Birinci katman — DTO Doğrulama (Data Annotations): Hospital.Shared projesindeki DTO sınıflarında [Required], [MaxLength], [StringLength], [Range] ve [EmailAddress] gibi attribute'lar tanımlanmıştır. Bu attribute'lar Validator.TryValidateObject() metodu aracılığıyla sunucu tarafında kontrol edilir.

İkinci katman — API Doğrulama (RequestValidator): HospitalApi'de her POST ve PUT endpoint'i, request nesnesini RequestValidator üzerinden geçirir. Doğrulama başarısız olursa HTTP 400 Bad Request ve hata detayları döner.

Üçüncü katman — MAUI İstemci Doğrulama: MAUI sayfalarında kritik alanlar (zorunlu ad, T.C. kimlik formatı, şifre eşleşmesi vb.) istek API'ye gönderilmeden önce kod tarafında kontrol edilir. Hata varsa DisplayAlert diyalogu kullanıcıya gösterilir; API isteği yapılmaz.

Kod: src/Hospital.Shared/Dtos/PatientDtos.cs – Data Annotations

```
public class CreatePatientRequest
{
    [Required(ErrorMessage = "Ad zorunludur.")]
    [MaxLength(100)]
    5 başvuru
    public string FirstName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Soyad zorunludur.")]
    [MaxLength(100)]
    5 başvuru
    public string LastName { get; set; } = string.Empty;

    [Required(ErrorMessage = "T.C. kimlik numarası zorunludur.")]
    [StringLength(11, MinimumLength = 11, ErrorMessage = "T.C. kimlik 11 haneli olmalıdır.")]
    7 başvuru
    public string NationalId { get; set; } = string.Empty;

    [Required(ErrorMessage = "Telefon zorunludur.")]
    [MaxLength(20)]
    5 başvuru
    public string Phone { get; set; } = string.Empty;

    [EmailAddress(ErrorMessage = "Geçerli bir e-posta giriniz.")]
    [MaxLength(200)]
    7 başvuru
    public string? Email { get; set; }

    [Required]
    5 başvuru
    public DateTime BirthDate { get; set; }
```

Kod:

src/HospitalApi/Validation/RequestValidator.cs

```
1  using System.ComponentModel.DataAnnotations;
2
3  namespace HospitalApi.Validation;
4
5  21 başvuru
6  public static class RequestValidator
7  {
8      21 başvuru
9      public static IResult? Validate(object instance)
10     {
11         var results = new List<ValidationResult>();
12         var context = new ValidationContext(instance);
13         if (Validator.TryValidateObject(instance, context, results, validateAllProperties: true))
14         {
15             return null;
16         }
17
18         var errors = results
19             .Where(r => !string.IsNullOrEmpty(r.ErrorMessage))
20             .GroupBy(r => r.MemberNames.FirstOrDefault() ?? string.Empty)
21             .ToDictionary(
22                 g => string.IsNullOrEmpty(g.Key) ? "General" : g.Key,
23                 g => g.Select(x => x.ErrorMessage!).Distinct().ToArray());
24
25         return Results.ValidationProblem(errors);
26     }
```

Kod: src/Hospital.Client/PatientEditPage.xaml.cs – MAUI istemci doğrulama

```
private async void OnSaveClicked(object? sender, EventArgs e)
{
    var useNewPatient = NewPatientSwitch.IsToggled;
    if (!useNewPatient && (PatientPicker.SelectedIndex < 0 || PatientPicker.SelectedIndex >= _patientIds.Count)
    {
        await DisplayAlert("Doğrulama", "Hasta seçiniz veya yeni hasta seçeneğini açın.", "Tamam");
        return;
    }

    if (ClinicPicker.SelectedIndex < 0 || ClinicPicker.SelectedIndex >= _clinicIds.Count)
    {
        await DisplayAlert("Doğrulama", "Poliklinik seçiniz.", "Tamam");
        return;
    }

    if (DoctorPicker.SelectedIndex < 0 || DoctorPicker.SelectedIndex >= _doctorIds.Count)
    {
        await DisplayAlert("Doğrulama", "Bu poliklinik için doktor bulunamadı veya seçilmedi.", "Tamam");
        return;
    }
}
```

İSTER 23 – LINQ Aktif Kullanımı

LINQ (Language Integrated Query), projede servis katmanındaki tüm veri sorgulama, filtreleme, sıralama ve dönüştürme işlemlerinde kullanılmaktadır. EF Core, LINQ sorgularını SQL sorgularına çevirerek doğrudan veritabanında yürütür; bu sayede gereksiz veri uygulama belleğine aktarılmaz.

Kullanılan LINQ operatörleri ve amaçları:

- **Where:** Filtreleme koşulları zinciri kurmak için
- **Select:** Entity sınıflarını DTO sınıflarına dönüştürmek için (projeksiyon)
- **OrderByDescending:** En yeni kayıtları üste almak için oluşturma tarihine göre ters sıralama
- **AsNoTracking:** Performans: değişiklik izleyiciyi devre dışı bırakır (read-only sorgular)
- **AsQueryable:** Dinamik sorgu zinciri kurmak için (koşula göre Where ekleme)
- **ToListAsync:** Sonuçları listeye dönüştürür ve SQL sorgusunu çalıştırır
- **AnyAsync:** Kayıt var mı kontrolü — COUNT(*) yerine EXISTS kullanır
- **FirstOrDefaultAsync:** İlk eşleşen kayıt veya null döner
- **Include:** Navigation property'leri (ilişkili tablolar) eager loading ile çeker

Kod: src/HospitalApi/Services/PatientService.cs – tam LINQ zinciri

```
3 Başvuru
public async Task<IReadOnlyList<PatientDto>> GetAsync(PatientListQuery query,
{
    var linq = db.Patients.AsNoTracking().AsQueryable();

    if (!string.IsNullOrEmpty(query.Search))
    {
        var term = query.Search.Trim();
        linq = linq.Where(p =>
            p.FirstName.Contains(term) ||
            p.LastName.Contains(term) ||
            (p.Email != null && p.Email.Contains(term)) ||
            p.Phone.Contains(term));
    }

    if (!string.IsNullOrEmpty(query.NationalId))
    {
        var nid = query.NationalId.Trim();
        linq = linq.Where(p => p.NationalId == nid);
    }

    return await linq
        .OrderByDescending(p => p.CreatedAt)
        .Select(p => new PatientDto
        {
            Id = p.Id,
            FirstName = p.FirstName,
            LastName = p.LastName,
            NationalId = p.NationalId,
            Phone = p.Phone,
            Email = p.Email,
            BirthDate = p.BirthDate,
            EmergencyContactName = p.EmergencyContactName,
            EmergencyContactPhone = p.EmergencyContactPhone,
            EmergencyContactRelation = p.EmergencyContactRelation,
            CreatedAt = p.CreatedAt,
            CreatedByUserId = p.CreatedByUserId,
            UpdatedAt = p.UpdatedAt,
            UpdatedByUserId = p.UpdatedByUserId
        })
        .ToListAsync(cancellationToken);
}
```

Kod: AppointmentService.cs – çoklu tablo sorgusu (Include + Where + Select)

```
4 Başvuru
public async Task<IReadOnlyList<AppointmentDto>> GetAsync(AppointmentListQuery query,
{
    var linq = from a in db.Appointments.AsNoTracking()
                join p in db.Patients.AsNoTracking() on a.PatientId equals p.Id
                join d in db.Doctors.AsNoTracking() on a.DoctorId equals d.Id
                join c in db.Clinics.AsNoTracking() on a.ClinicId equals c.Id
                select new { Appointment = a, Patient = p, Doctor = d, Clinic = c };

    if (query.PatientId is int pid && pid > 0)
    {
        linq = linq.Where(x => x.Appointment.PatientId == pid);
    }

    if (query.DoctorId is int did && did > 0)
    {
        linq = linq.Where(x => x.Appointment.DoctorId == did);
    }

    if (query.ClinicId is int cid && cid > 0)
    {
        linq = linq.Where(x => x.Appointment.ClinicId == cid);
    }

    if (query.From.HasValue)
    {
        var from = query.From.Value;
        linq = linq.Where(x => x.Appointment.ScheduledAt >= from);
    }

    if (query.To.HasValue)
    {
        var to = query.To.Value;
        linq = linq.Where(x => x.Appointment.ScheduledAt <= to);
    }

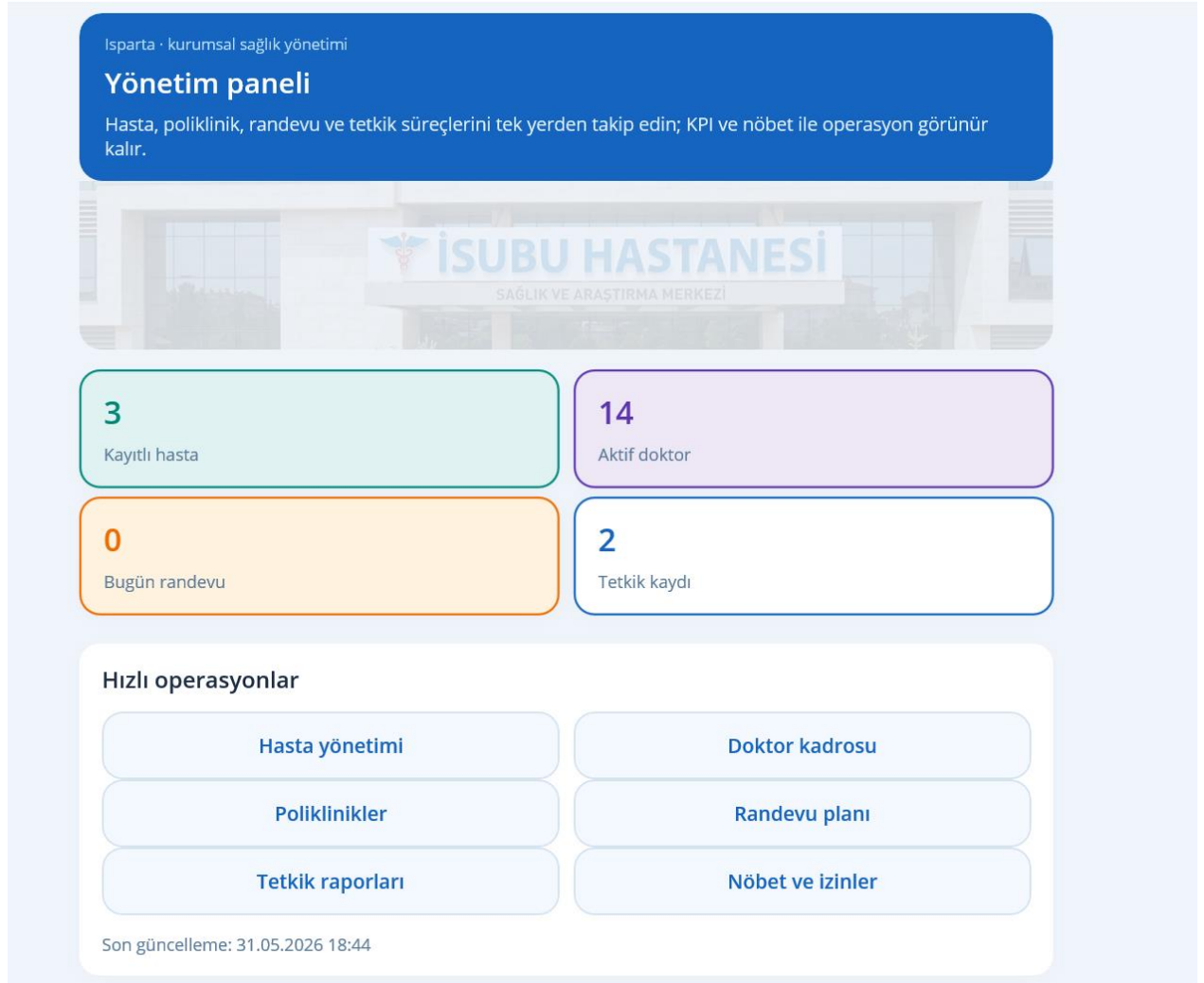
    if (query.Status.HasValue)
    {
        var status = query.Status.Value;
        linq = linq.Where(x => x.Appointment.Status == status);
    }

    return await linq
        .OrderByDescending(x => x.Appointment.ScheduledAt)
        .Select(x => new AppointmentDto
        {
            Id = x.Appointment.Id
        })
        .ToListAsync();
}
```

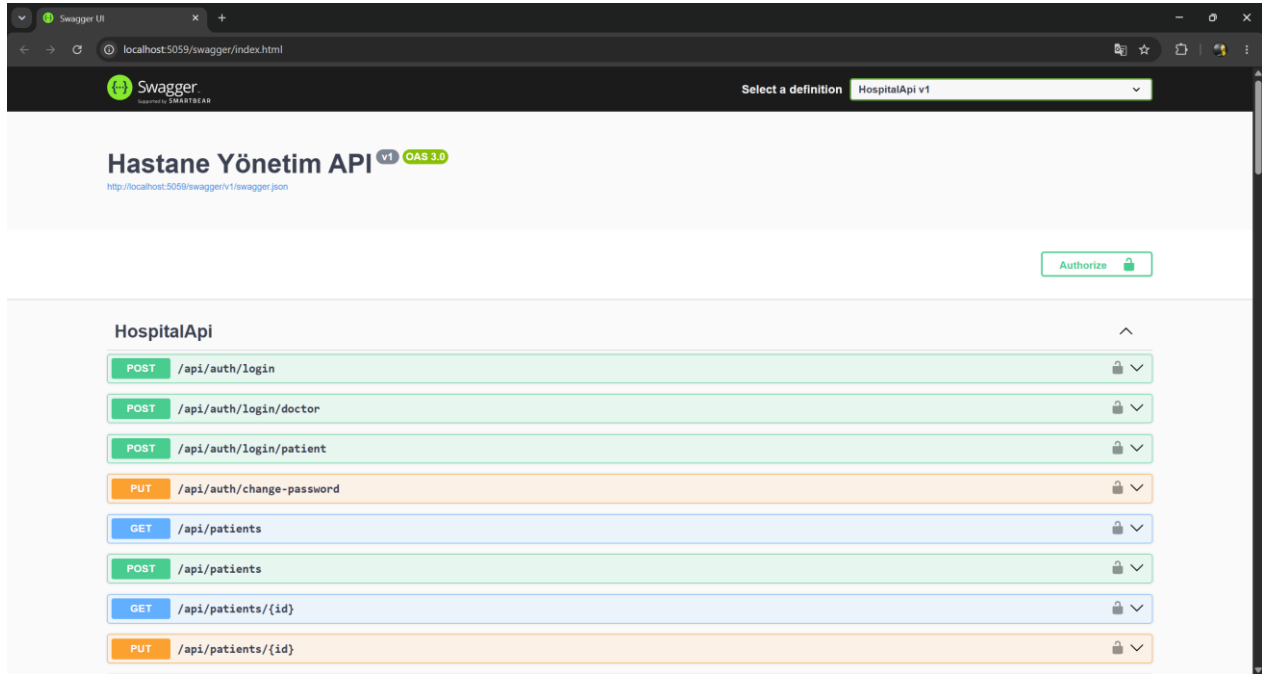
İSTER 24 – Proje Çalışıyor mu?

Proje başarıyla derlenmiş ve çalışır durumdadır. Hospital.sln çözüm dosyası üç projeyi bir arada yönetir: Hospital.Client (MAUI), HospitalApi (API) ve Hospital.Shared (DTO). API, SQLite profili ile tek komutla başlatılabilir; MAUI uygulaması Windows 11'de yerel uygulama olarak çalıştırılmıştır.

API <http://localhost:5059> adresinde hizmet vermekte, MAUI uygulaması bu adrese HTTP istekleri göndermektedir. Swagger UI </swagger> adresinden erişilebilir olup tüm endpoint'lerin interaktif testine olanak tanır. Veritabanı, ilk çalıştırmada DbInitializer.SeedAsync() tarafından oluşturulur ve demo verilerle doldurulur: admin kullanıcısı (admin / Admin123!), örnek doktorlar, klinikler, hastalar ve randevular otomatik oluşturulur.



Şekil18: Uygulama çalışıp yönetim paneli açıldığında veritabanına kayıtlı olan tüm işlemler görünür.



Şekil19: SwaggerUI

Build Çıktısı (src/HospitalApi)

```
C:\Users\eftal\Desktop\isubu_hastane>dotnet build src/HospitalApi/HospitalApi.csproj
0,3sn içinde 1 uyarıyla başarılı oldu geri yükle
C:\Users\eftal\Desktop\isubu_hastane\src\HospitalApi\HospitalApi.csproj : warning NU1510: PackageReference Microsoft.Extensions.Identity.Core ayıklanmayacak. Bu paketi bağımlılıklarınızdan kaldırmayı düşünün, çünkü muhtemelen gereksizdir.
Hospital.Shared net10.0 başarılı(0,1sn) → src\Hospital.Shared\bin\Debug\net10.0\Hospital.Shared.dll
HospitalApi net10.0 1 uyarıyla başarılı oldu(0,2sn) → src\HospitalApi\bin\Debug\net10.0\HospitalApi.dll
C:\Users\eftal\Desktop\isubu_hastane\src\HospitalApi\HospitalApi.csproj : warning NU1510: PackageReference Microsoft.Extensions.Identity.Core ayıklanmayacak. Bu paketi bağımlılıklarınızdan kaldırmayı düşünün, çünkü muhtemelen gereksizdir.
"1,0" sn'de 2 uyarıyla başarılı oldu oluşturun
```

API Başlatma (SQLite profili)

```
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5059
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\eftal\Desktop\isubu_hastane\src\HospitalApi
```

İSTER 25 – Hastane Konusuna Uygun Minimum İşlemler

Proje, gerçek bir hastanede yürütülen temel süreçlerin dijital karşılığını sunmaktadır. Yalnızca minimum şartları değil; hasta portalı, tetkik raporları, nöbet/izin yönetimi ve zaman çizelgesi gibi kapsamlı modülleri de barındırmaktadır.

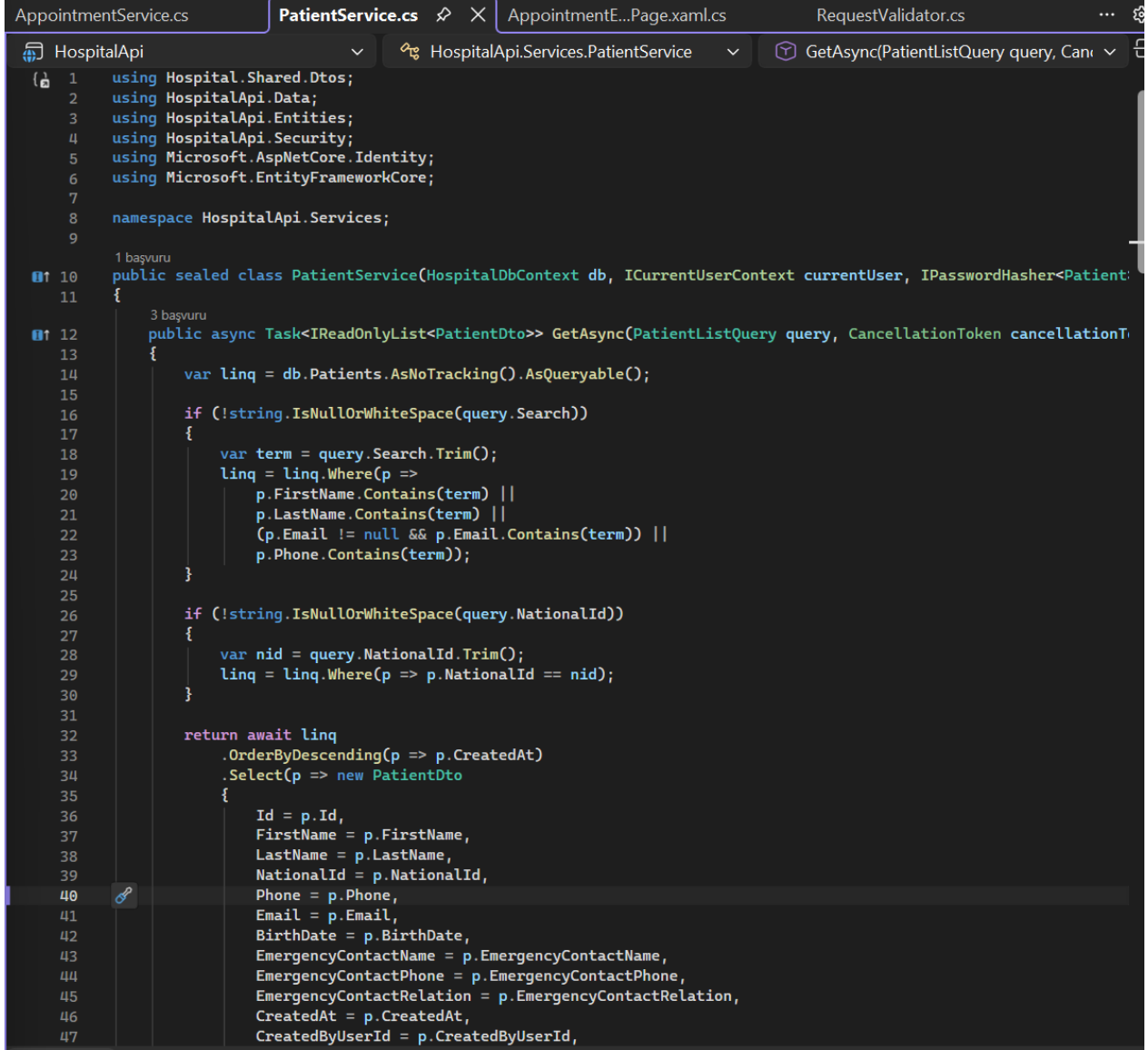
Modüller ve Özellikler

- **Hasta Yönetimi:** Yeni hasta kaydı oluşturma, mevcut hastayı güncelleme/silme, ad-soyad/TC/telefon/e-posta ile arama ve listeleme
- **Klinik (Poliklinik) Yönetimi:** Poliklinik ekleme, güncelleme, silme; klinik numarası ve açıklama yönetimi
- **Doktor Yönetimi:** Doktor kadrosu CRUD; klinik ataması, uzmanlık alanı ve iletişim bilgileri yönetimi
- **Randevu Yönetimi:** Randevu oluşturma (klinik→doktor→tarih→slot zinciri), listeleme, güncelleme, iptal; durum takibi (Planlandı/Tamamlandı/İptal)
- **Tetkik (Lab) Raporu Modülü:** Tetkik sonuç raporu ekleme, görüntüleme ve PDF dosyası yükleme/indirme
- **Nöbet / İzin Yönetimi:** Doktor nöbet ve izin kayıtlarının oluşturulması, listelenmesi ve düzenlenmesi
- **Hasta Sağlık Zaman Çizelgesi:** Hastanın tüm randevularını ve tetkik raporlarını kronolojik zaman çizelgesi olarak görüntülemesi
- **Üç Katmanlı Portal (Admin / Doktor / Hasta):** Admin / Doktor / Hasta için ayrı shell, giriş ekranı ve yetki alanı; rol bazlı endpoint erişim kontrolü
- **Güvenli Kimlik Doğrulama:** Sistem yöneticisi kullanıcı adı/şifre ile, doktorlar T.C./şifre ile, hastalar T.C./şifre ile sisteme girer; JWT token 4 saat geçerlidir

İSTER 26 – C# İsimlendirme Kurallarına Uyum

Projede Microsoft'un resmi C# isimlendirme kuralları eksiksiz uygulanmıştır. Bu kurallar; kodun okunabilirliğini artırır, takım içinde tutarlılık sağlar ve tooling (IDE desteği, analiz araçları) ile uyumlu çalışmayı kolaylaştırır.

Kod: src/HospitalApi/Services/PatientService.cs – kuralların bir arada görüldüğü örnek



```
1 using Hospital.Shared.Dtos;
2 using HospitalApi.Data;
3 using HospitalApi.Entities;
4 using HospitalApi.Security;
5 using Microsoft.AspNetCore.Identity;
6 using Microsoft.EntityFrameworkCore;
7
8 namespace HospitalApi.Services;
9
10 public sealed class PatientService(HospitalDbContext db, ICurrentUserContext currentUser, IPasswordHasher<Patient>
11 {
12     public async Task<IEnumerable<PatientDto>> GetAsync(PatientListQuery query, CancellationToken cancellationT
13     {
14         var linq = db.Patients.AsNoTracking().AsQueryable();
15
16         if (!string.IsNullOrEmpty(query.Search))
17         {
18             var term = query.Search.Trim();
19             linq = linq.Where(p =>
20                 p.FirstName.Contains(term) ||
21                 p.LastName.Contains(term) ||
22                 (p.Email != null && p.Email.Contains(term)) ||
23                 p.Phone.Contains(term));
24         }
25
26         if (!string.IsNullOrEmpty(query.NationalId))
27         {
28             var nid = query.NationalId.Trim();
29             linq = linq.Where(p => p.NationalId == nid);
30         }
31
32         return await linq
33             .OrderByDescending(p => p.CreatedAt)
34             .Select(p => new PatientDto
35             {
36                 Id = p.Id,
37                 FirstName = p.FirstName,
38                 LastName = p.LastName,
39                 NationalId = p.NationalId,
40                 Phone = p.Phone,
41                 Email = p.Email,
42                 BirthDate = p.BirthDate,
43                 EmergencyContactName = p.EmergencyContactName,
44                 EmergencyContactPhone = p.EmergencyContactPhone,
45                 EmergencyContactRelation = p.EmergencyContactRelation,
46                 CreatedAt = p.CreatedAt,
47                 CreatedByUserId = p.CreatedByUserId,
```

```
HospitalApi
HospitalApi.Services.PatientService
GetAsync(PatientListQuery query, Canc...

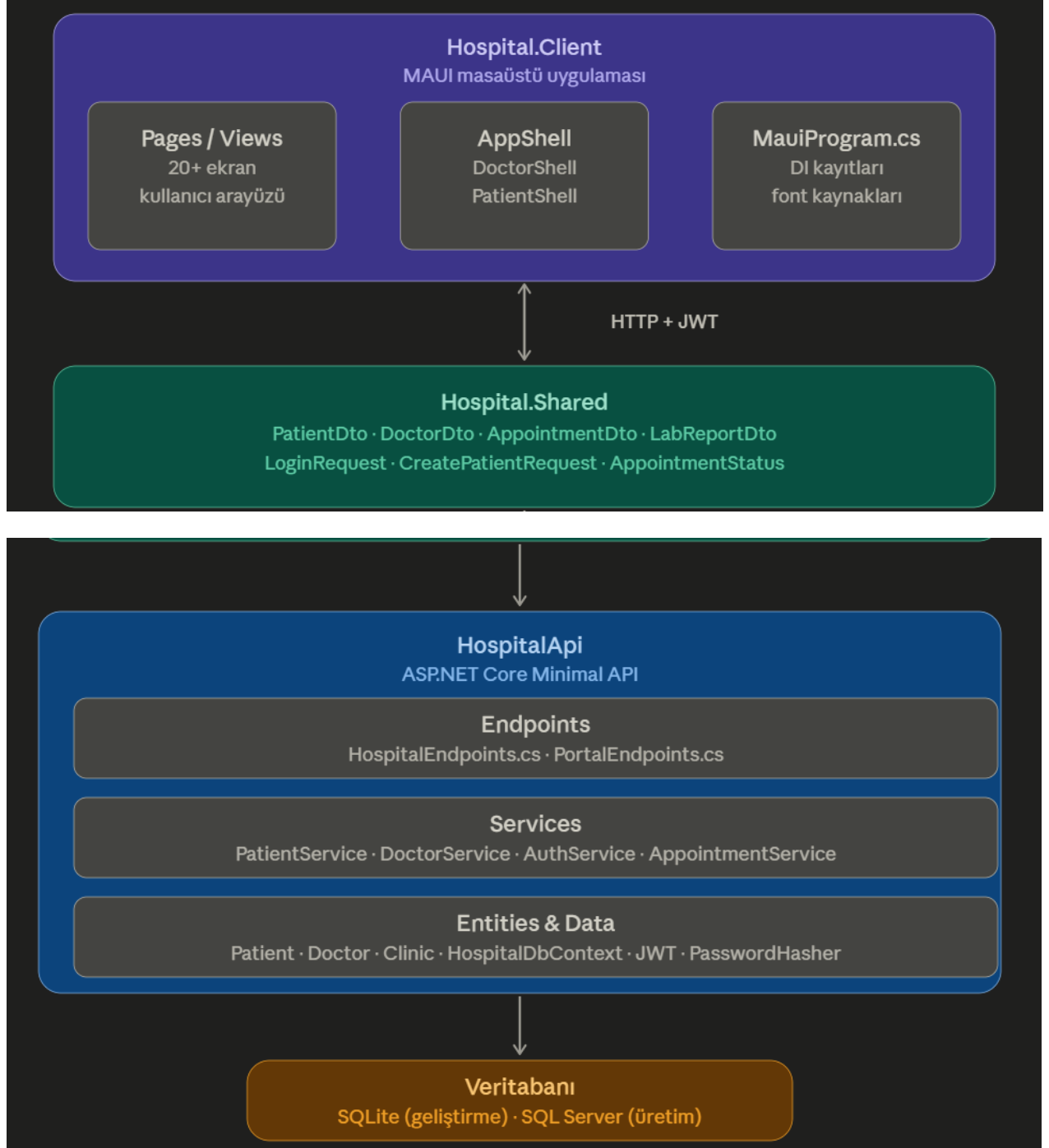
53
54 public async Task<PatientDto?> GetByIdAsync(int id, CancellationToken cancellationToken = default)
55 {
56     return await db.Patients.AsNoTracking()
57         .Where(p => p.Id == id)
58         .Select(p => new PatientDto
59         {
60             Id = p.Id,
61             FirstName = p.FirstName,
62             LastName = p.LastName,
63             NationalId = p.NationalId,
64             Phone = p.Phone,
65             Email = p.Email,
66             BirthDate = p.BirthDate,
67             EmergencyContactName = p.EmergencyContactName,
68             EmergencyContactPhone = p.EmergencyContactPhone,
69             EmergencyContactRelation = p.EmergencyContactRelation,
70             CreatedAt = p.CreatedAt,
71             CreatedByUserId = p.CreatedByUserId,
72             UpdatedAt = p.UpdatedAt,
73             UpdatedByUserId = p.UpdatedByUserId
74         })
75         .FirstOrDefaultAsync(cancellationToken);
76 }
77

2 başvuru
78 public async Task<(bool Success, string? Error, PatientDto? Patient)> CreateAsync(CreatePatientRequest request)
79 {
80     var exists = await db.Patients.AnyAsync(p => p.NationalId == request.NationalId, cancellationToken);
81     if (exists)
82     {
83         return (false, "Bu T.C. kimlik numarası ile kayıtlı hasta zaten var.", null);
84     }
85
86     var userId = currentUser.UserId;
87     var entity = new Patient
88     {
89         FirstName = request.FirstName.Trim(),
90         LastName = request.LastName.Trim(),
91         NationalId = request.NationalId.Trim(),
92         Phone = request.Phone.Trim(),
93         Email = string.IsNullOrEmpty(request.Email) ? null : request.Email.Trim(),
94         BirthDate = request.BirthDate.Date,
95         CreatedAt = DateTime.UtcNow,
96         CreatedByUserId = userId
97     };
98     entity.PortalPasswordHash = patientPasswordHasher.HashPassword(entity, "Patient123!");
99 }
```

Not: MAUI client tarafındaki tüm sayfalar (PatientsPage, PatientEditPage, LoginPage vb.), view modeller ve servis sınıfları aynı isimlendirme kurallarına uymaktadır.

3. Mimari Özet

Proje, sorumlulukları net biçimde ayrılmış katmanlı bir mimariye sahiptir. İstemci ve sunucu arasındaki iletişim HTTP + JWT ile sağlanır; paylaşılan modeller (DTO'lar) Hospital.Shared projesinde toplanmıştır.



Her katmanın tek bir sorumluluğu vardır: MAUI katmanı kullanıcı arayüzünü, API katmanı iş mantığını, EF Core/veritabanı katmanı ise veri kalıcılığını yönetir. Katmanlar arasındaki bağımlılıklar interface'ler ve DTO'lar aracılığıyla gevşek bağımlı (loosely coupled) tutulmuştur.